



Deep Neural Networks



Why do we need *Sequential Modeling*





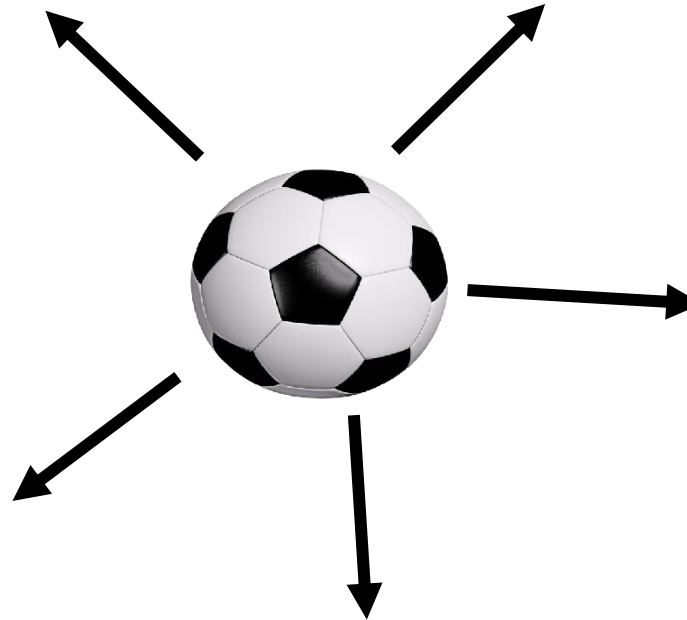
Why do we need Sequential Modeling

Given a Football Image





Why do we need Sequential Modeling



Given a Football Image

Can you predict where
it will go next?



Why do we need Sequential Modeling



Now can you predict?



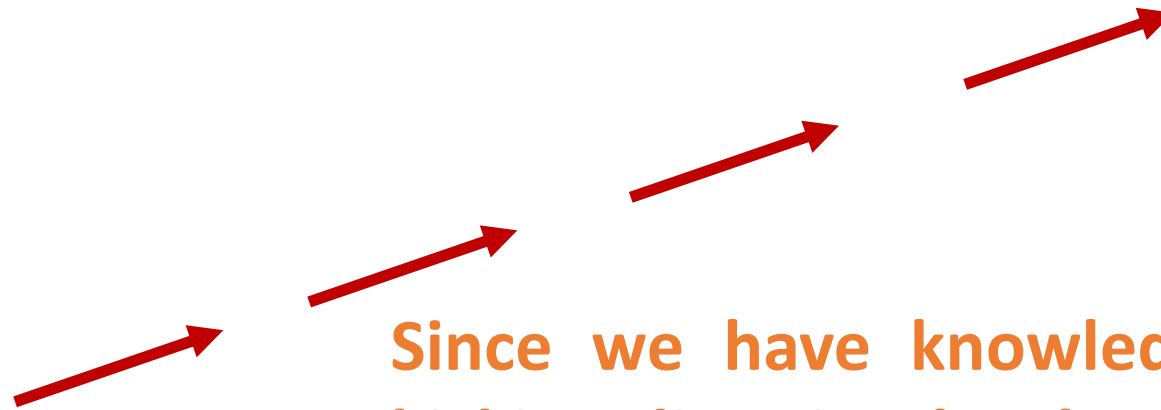
Why do we need Sequential Modeling



Since we have knowledge about the kicking direction by the player, we can predict its next direction.



Why do we need Sequential Modeling



Since we have knowledge about the kicking direction by the player, we can predict its next direction.



Need for Sequential Modeling

Sequence data

Speech recognition

Machine translation

Music generation

Name entity
recognition

Input data



You are my best friend

∅

GH Hardy said, his
contribution was
discovery of Ramanujan.

Output

Wow, it is so nice!

Você é meu melhor amigo



GH Hardy said, his
contribution was discovery
of Ramanujan.

Need for Sequential Modeling

Sequence data

Sentiment
classification

DNA sequence
analysis

Video activity
recognition

Input data

Wow, it is so nice!

```
A5ASC3.1
B4F917.1
A9S1V2.1
B9GSN7.1
Q8H056.1
Q0D4Z3.2
```



Output



```
SIKLWPPSQTTTRLLVERMANNLST..PSIFTRK.
SIKLWPPSESTRIMLVDRMTNNLST..ESIFSRK.
VFKLWPPSQGTREAVRQKMAKLSS..ACFESQS.
SVKLWPPGQSTRMLVERMTKNFIT..PSFISRK.
SFSIWPPPTQRTDRAVVRRLVDTLGG..DTILCKR.
LSIWPPSQRTDRAVVRRLVQTLVA..PSILSQR.
```

Fighting



Can we use ANN/CNN for *Sequential Modeling*?



Can we use ANN/CNN for *Sequential Modeling*?

No 



Reasons:

- Fixed input size

Example : image size



32x32

Reasons:

- Fixed input size

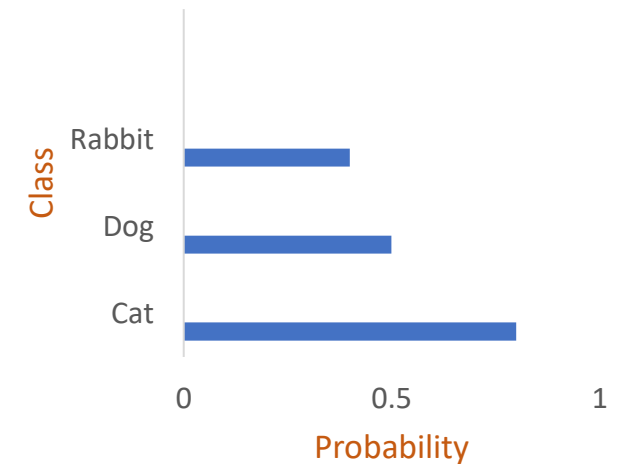
Example : image size



32x32

- Fixed output size

Example : probabilities of different classes



Reasons:

- Fixed input size

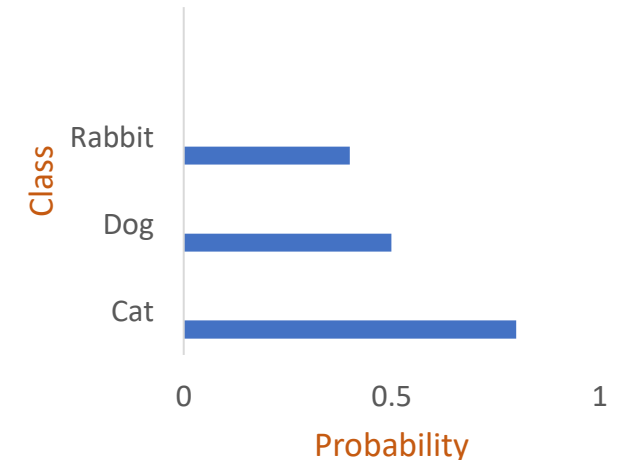
Example : **image size**



32x32

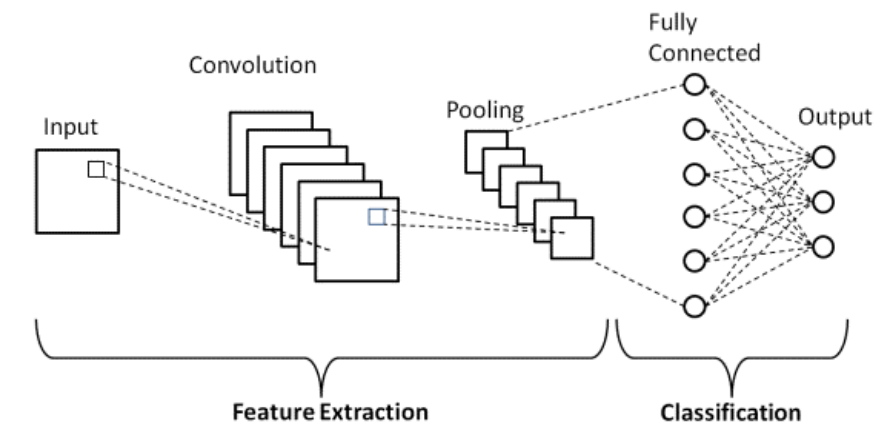
- Fixed output size

Example : **probabilities of different classes**



- Fixed computational steps

Example : **number of layers in the model**





Reasons:

- Words learned or approximated at a later position may change the approximation of a previous word.

Example :

- Blue dresses are looking good.
 - Blue dress is looking good.
- Parameter sharing is not done in conventional ANNs.



Reasons:

- Words learned or approximated at a later position may change the approximation of a previous word.

Example :

- Blue dresses are looking good.
 - Blue dress is looking good.
- Parameter sharing is not done in conventional ANNs.

There comes ***Recurrent Neural Network!***



Reasons for Using Recurrent Neural Network (RNN)

- Can handle inputs and outputs of **varying lengths**.





Reasons for Using Recurrent Neural Network (RNN)

- Can handle inputs and outputs of **varying lengths**.
- It involves **directed cycles** to recognize sequential characteristics of a data.





Reasons for Using Recurrent Neural Network (RNN)

- Can handle inputs and outputs of **varying lengths**.
- It involves **directed cycles** to recognize sequential characteristics of a data.
- **Shares parameters** across different parts of the network.





Reasons for Using Recurrent Neural Network (RNN)

- Can handle inputs and outputs of **varying lengths**.
- It involves **directed cycles** to recognize sequential characteristics of a data.
- **Shares parameters** across different parts of the network.
- Track **long-term** dependencies.





Reasons for Using Recurrent Neural Network (RNN)

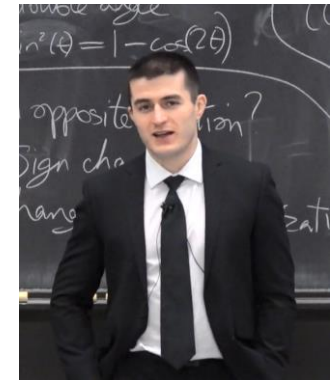
- Can handle inputs and outputs of **varying lengths**.
- It involves **directed cycles** to recognize sequential characteristics of a data.
- **Shares parameters** across different parts of the network.
- Track **long-term** dependencies.
- Maintain information about **order**.





When to use RNN?

“Whenever there is a sequence of data and the temporal dynamics that connects the data is more important than the spatial content of each individual frame.”

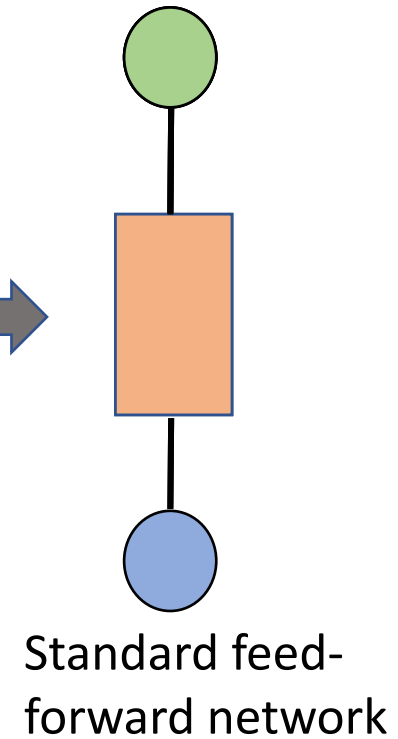
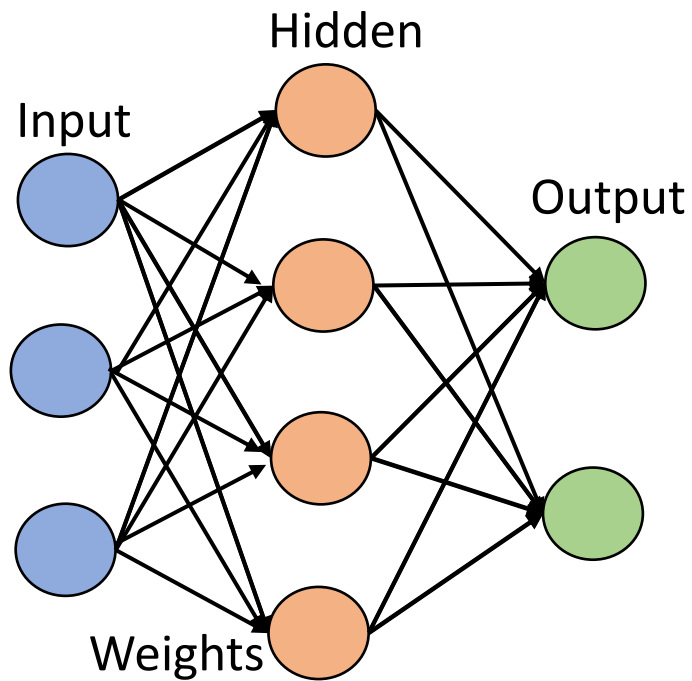


– Lex Fridman (MIT)

Image source: https://commons.wikimedia.org/wiki/File:Lex_Fridman_teaching_at_MIT_in_2018.png



Neural Network: Simplified



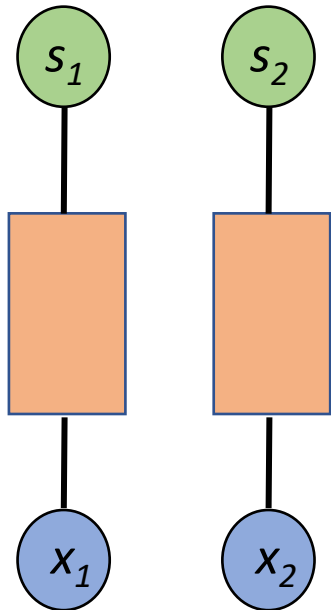


Handling Individual Time Steps



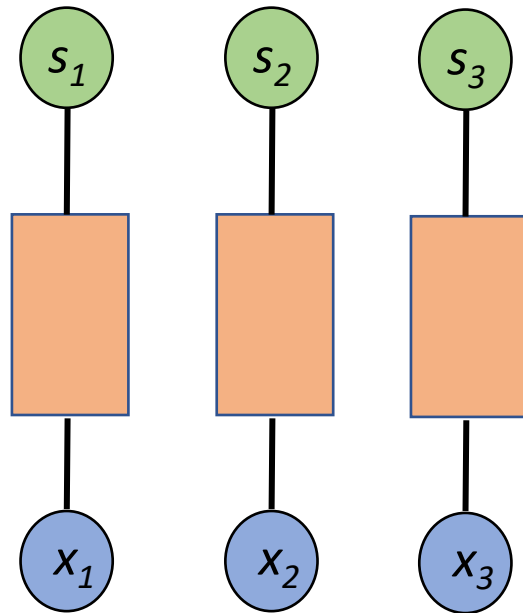


Handling Individual Time Steps

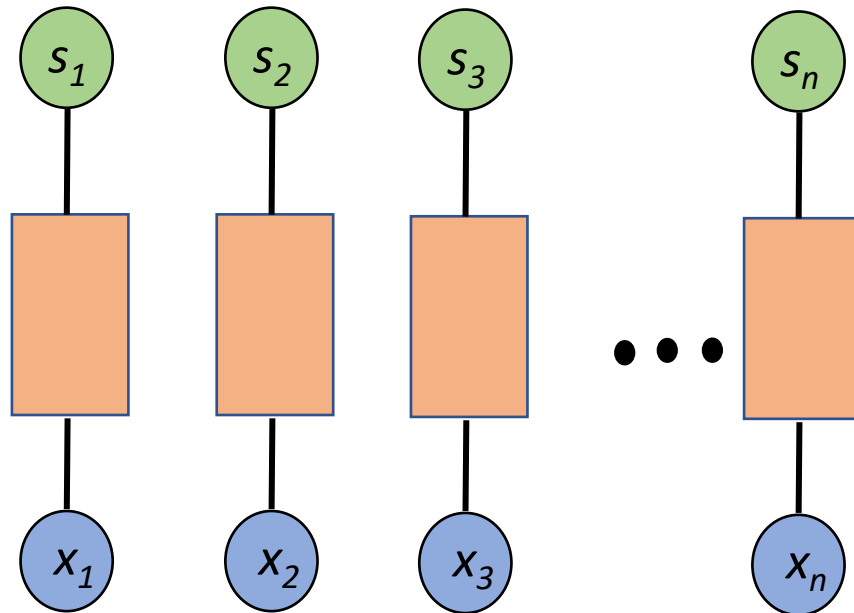




Handling Individual Time Steps



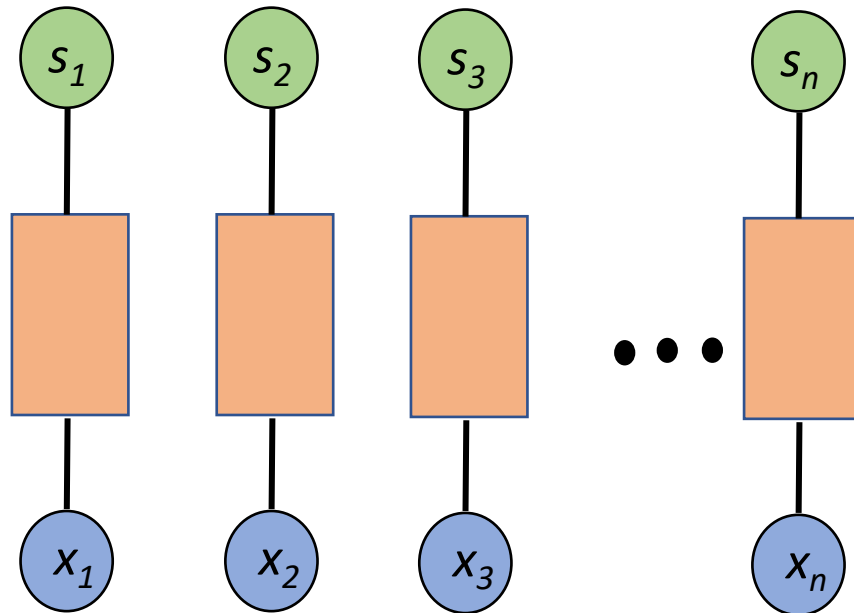
Handling Individual Time Steps



We can notice here that all these models are simply copying the same thing with different inputs at different time steps. We can make it concrete in terms of function transformation, since the predicted output is a function of input vector, hidden state, and the weight matrix



Handling Individual Time Steps



In general,

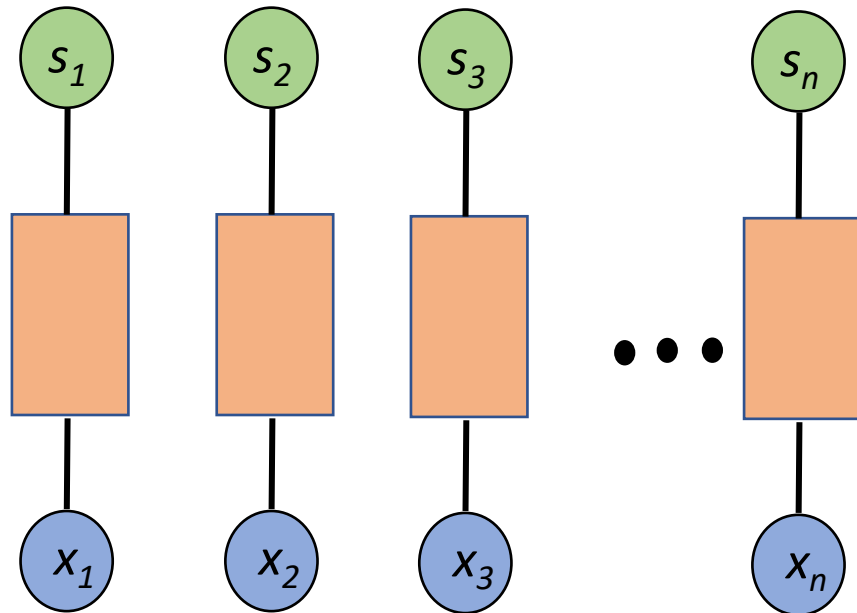
$$s_n = f(x_n)$$

n = time step

Note: We can make it concrete in terms of function transformation, since the predicted output is a function of input vector. The function f is learned by the weights of the network.



Handling Individual Time Steps



In general,

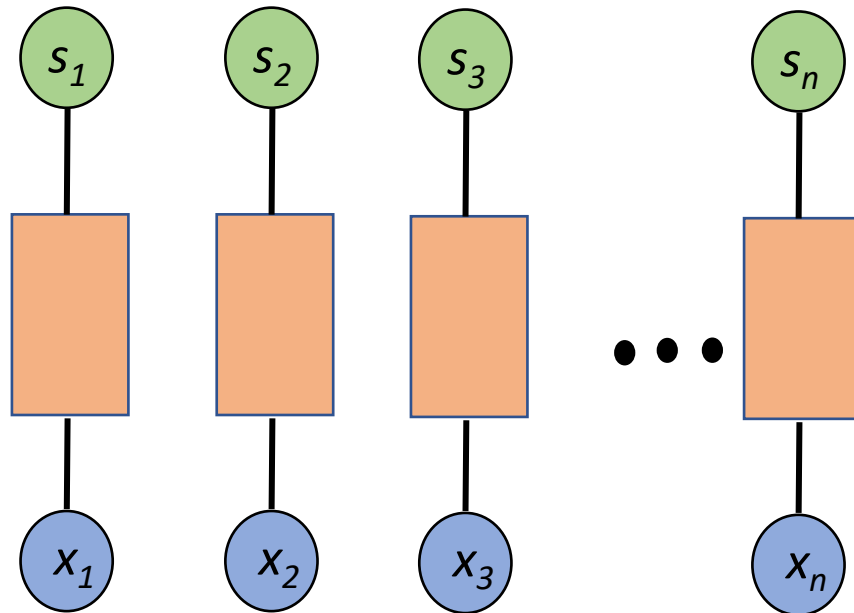
$$s_n = f(x_n)$$

$n = \text{time step}$

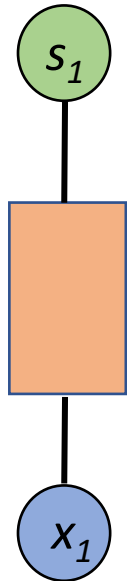
- Same function is used
- Replicate network any number of times
- Ensure parameter sharing
- Number of timestep does not matter



Handling Individual Time Steps



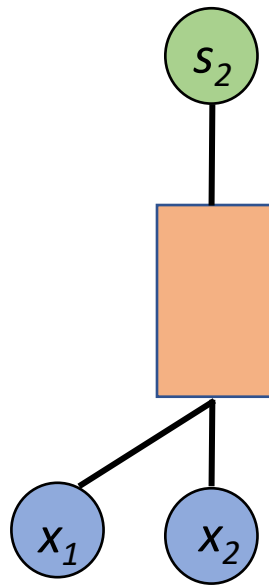
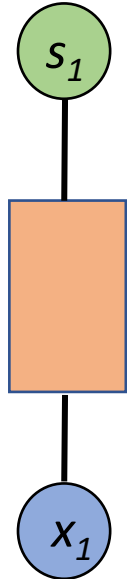
How to maintain the
interdependency between input?



A Simple Approach

Let's consider one approach





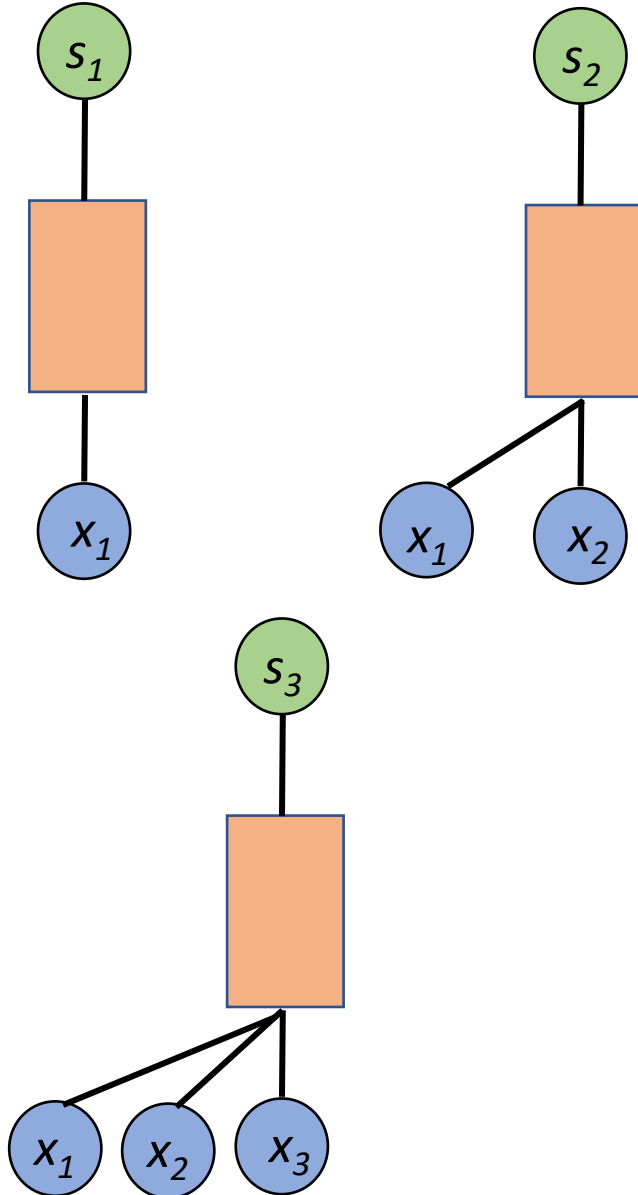
A Simple Approach

Let's consider one approach



A Simple Approach

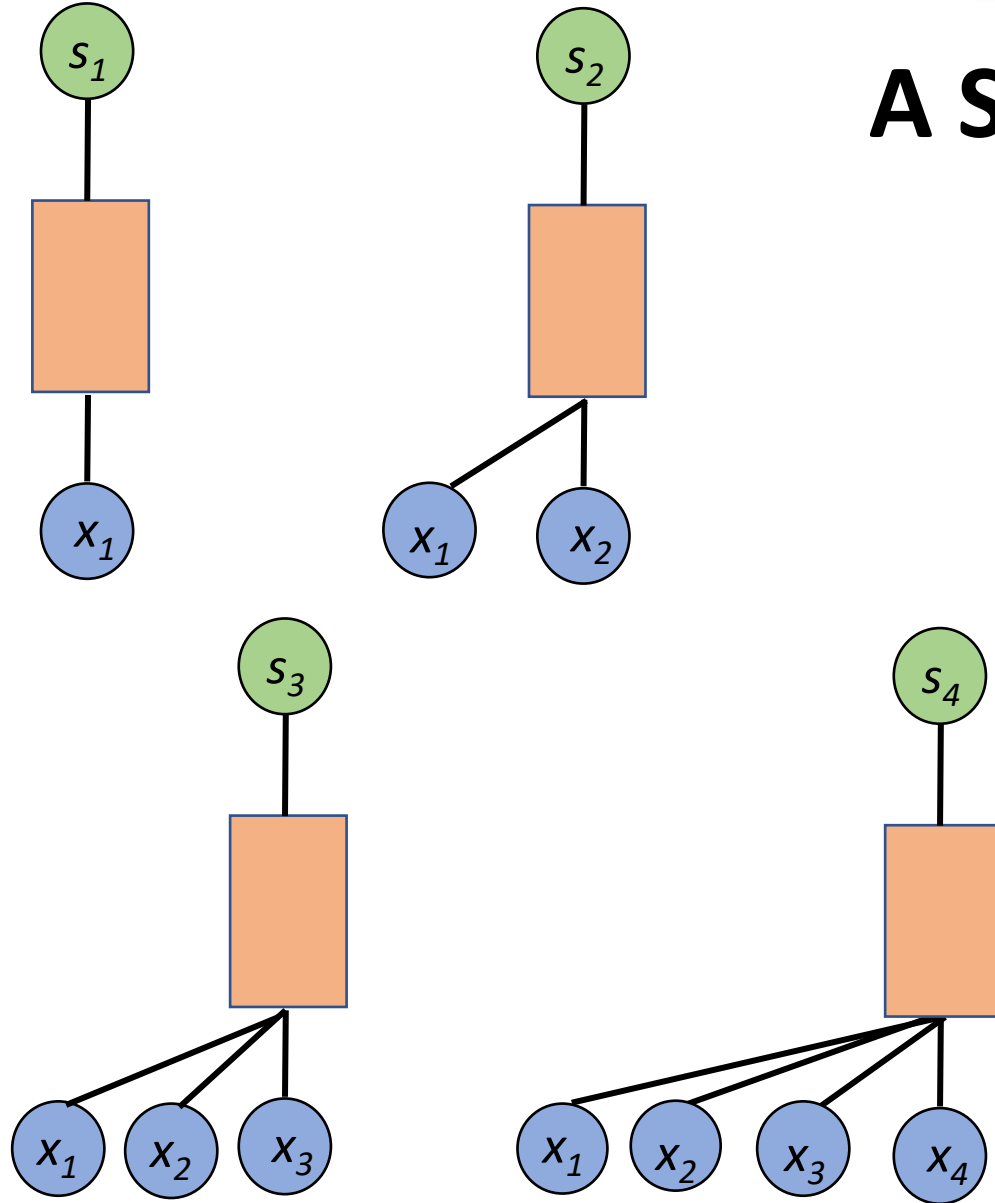
Let's consider one approach





A Simple Approach

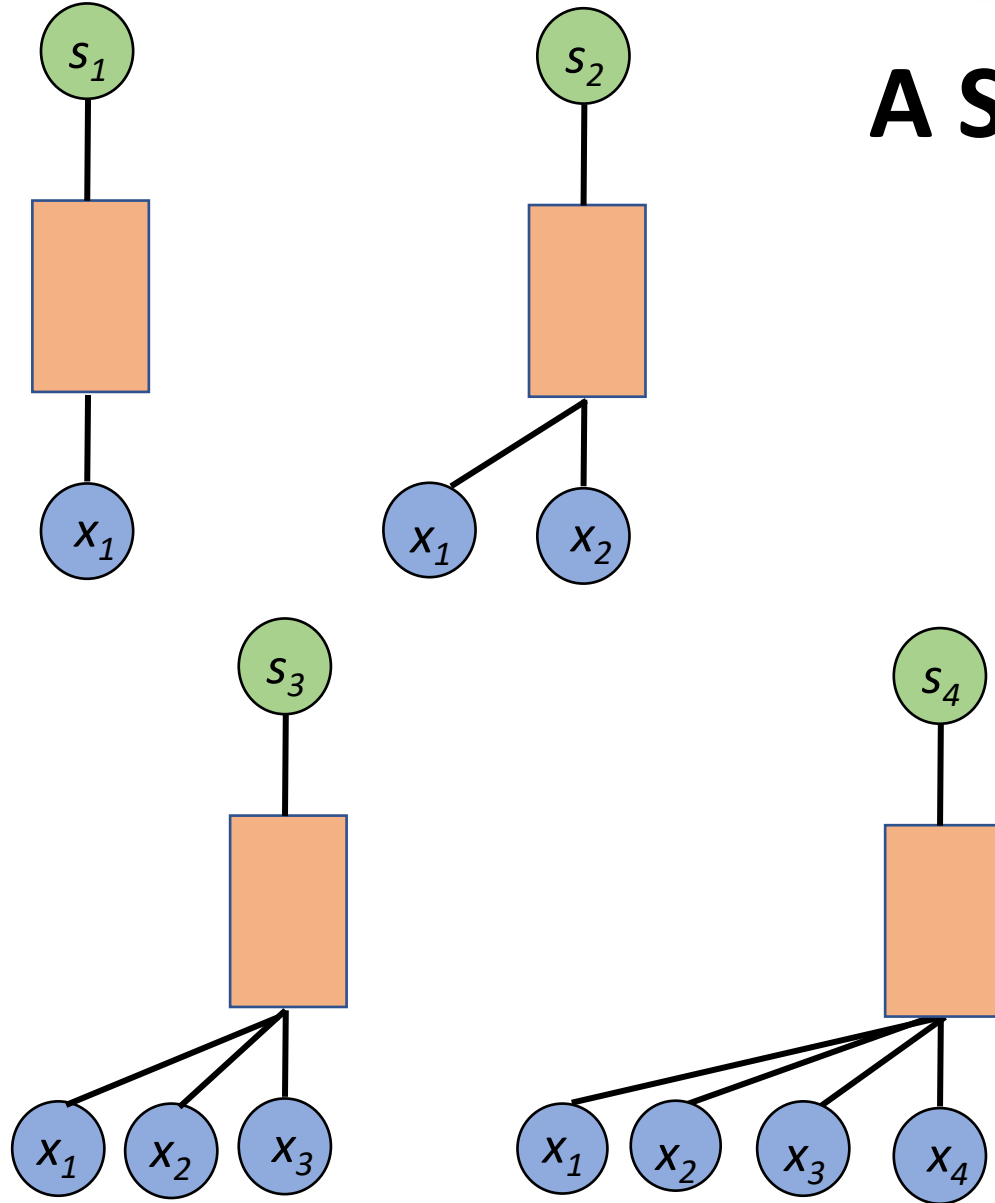
Let's consider one approach





A Simple Approach

Will this approach work?





A Simple Approach

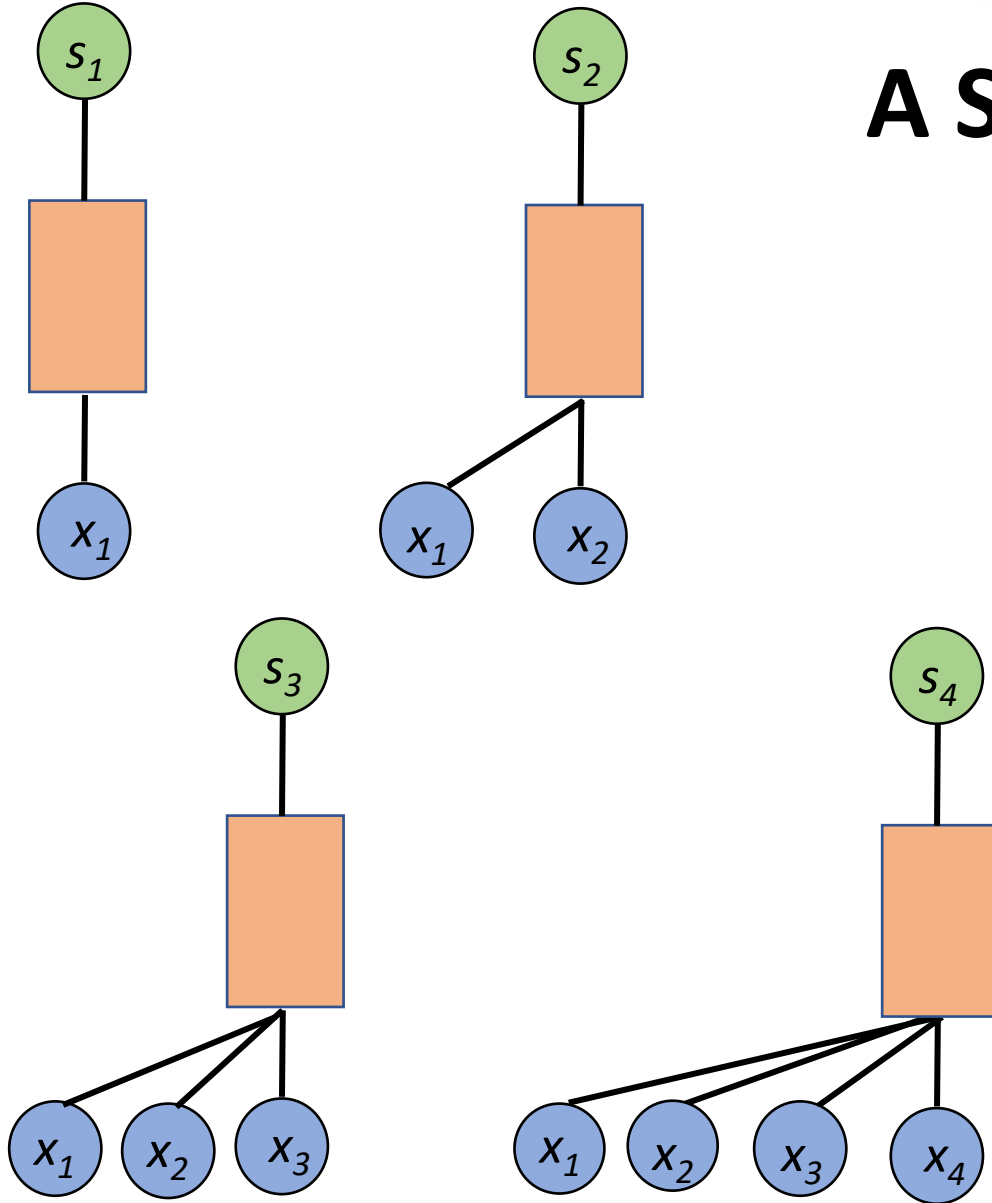
Problem

- Different function for different time-step

$$s_1 = f_1(x_1)$$

$$s_2 = f_2(x_1, x_2)$$

$$s_3 = f_3(x_1, x_2, x_3) \dots\dots$$





A Simple Approach

Problem

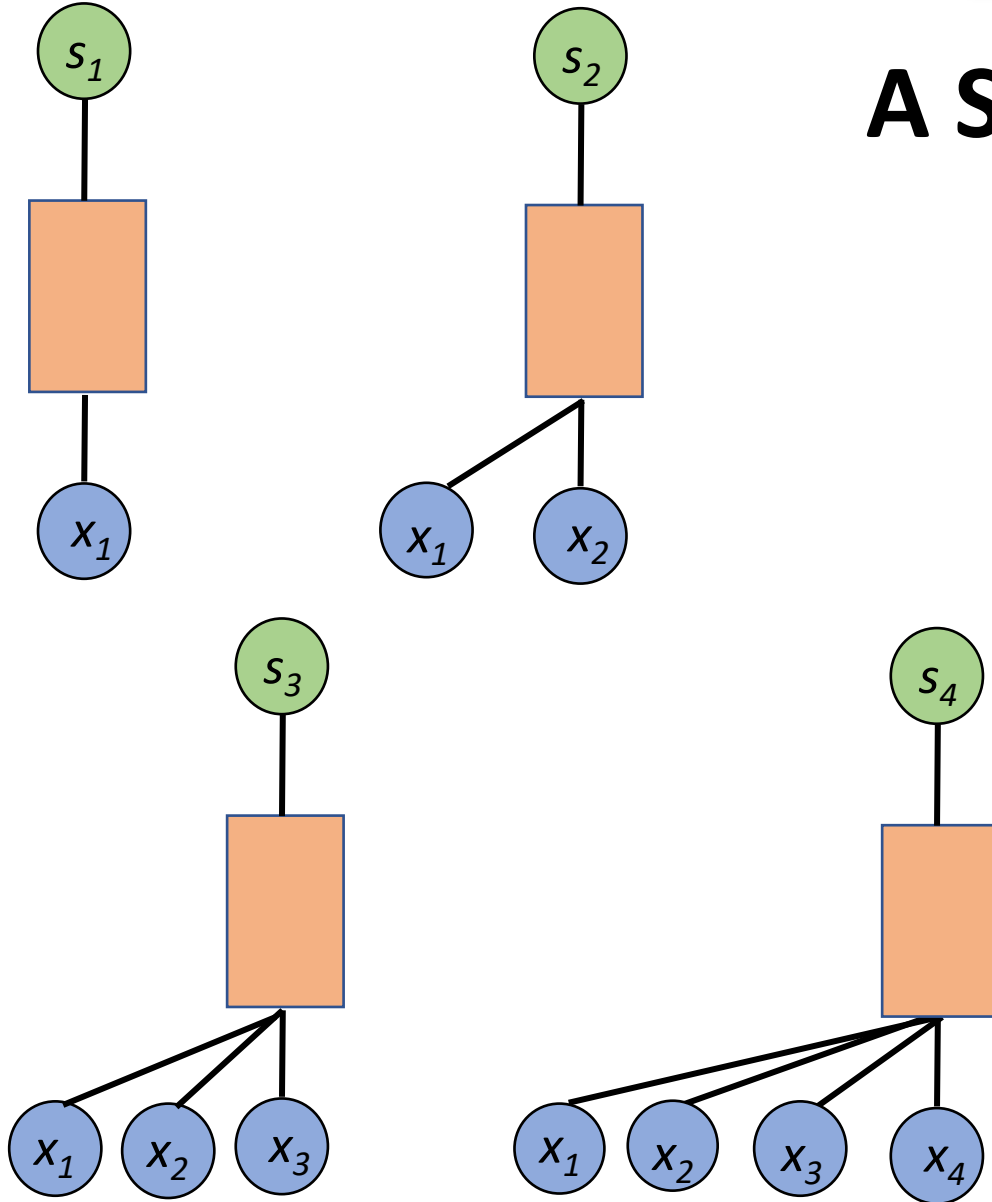
- Different function for different time-step

$$s_1 = f_1(x_1)$$

$$s_2 = f_2(x_1, x_2)$$

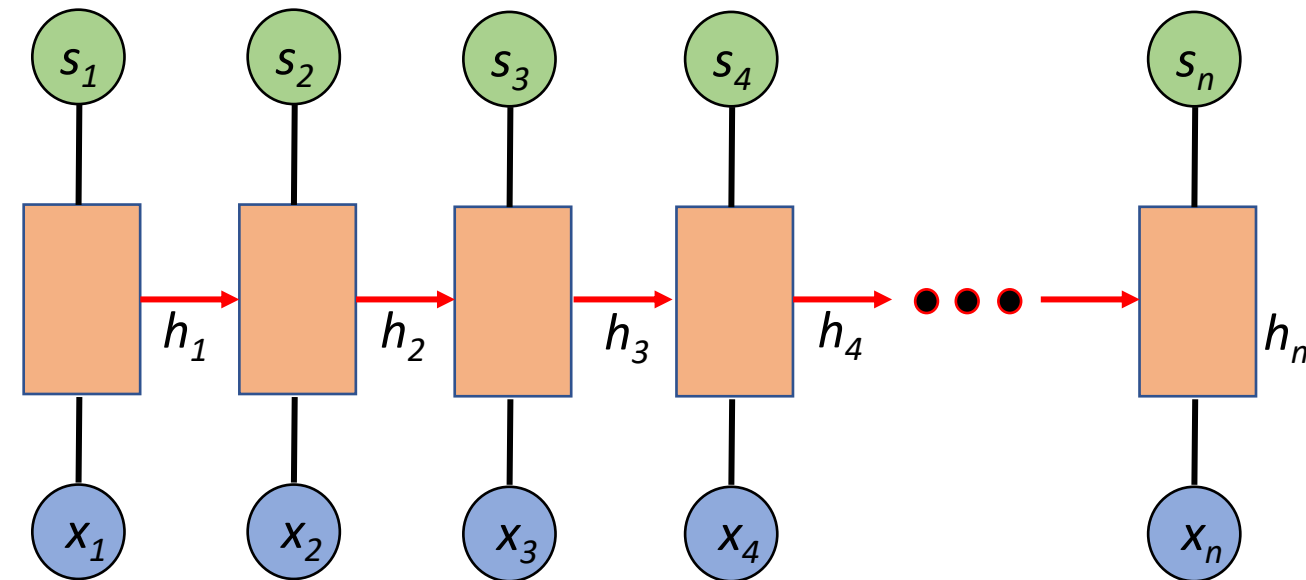
$$s_3 = f_3(x_1, x_2, x_3) \dots\dots$$

- Depends on input length





Recurrent Neural Network



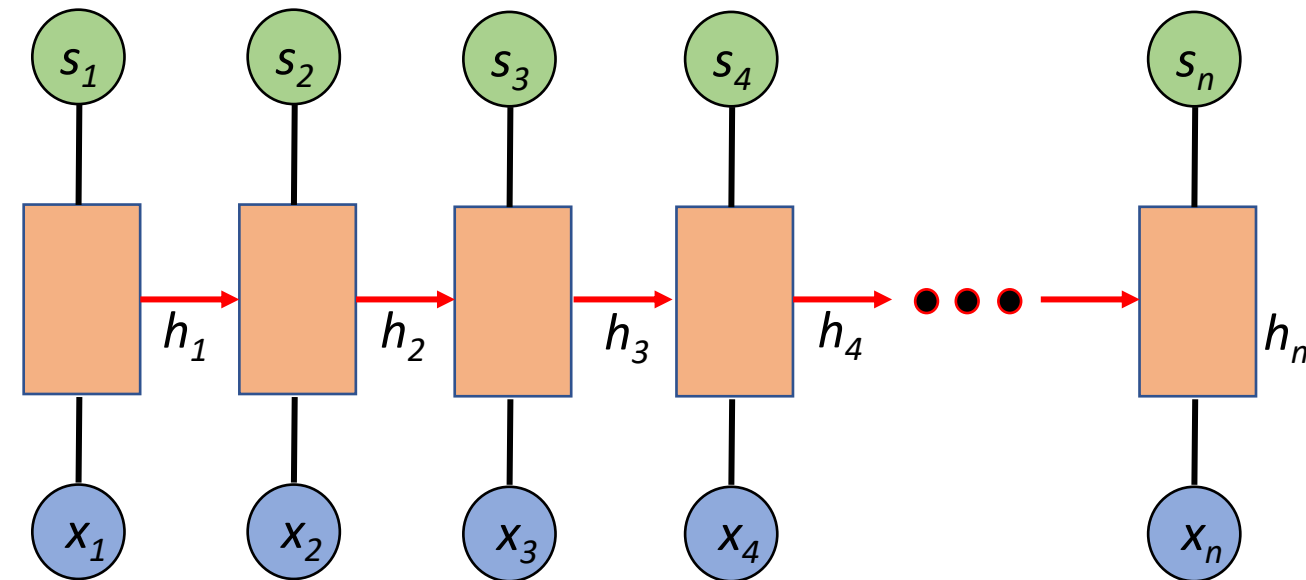
Solution

Add recurrent connection

Solution of interdependency is to add a recurrent connection in the network.
A sequence of vectors is processed by applying a recurrence formula at each time step.



Recurrent Neural Network

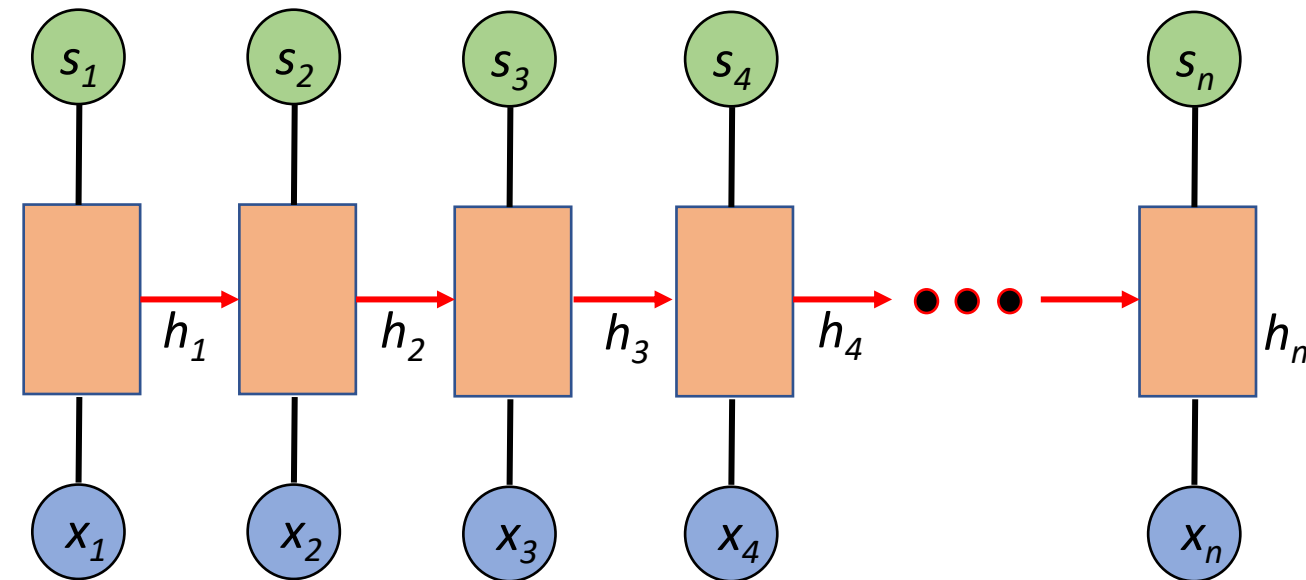


Solution

Add recurrent connection

$$s_n = f(x_n, h_{n-1})$$

Recurrent Neural Network



Solution

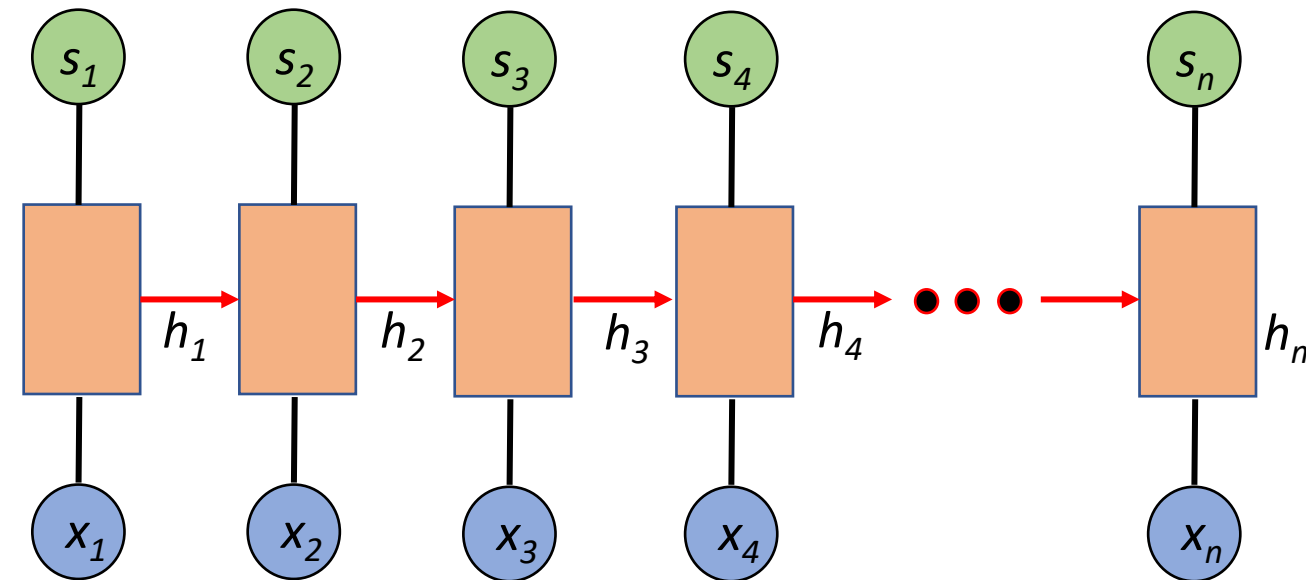
Add recurrent connection

$$s_n = f(x_n, h_{n-1})$$

input



Recurrent Neural Network



Solution

Add recurrent connection

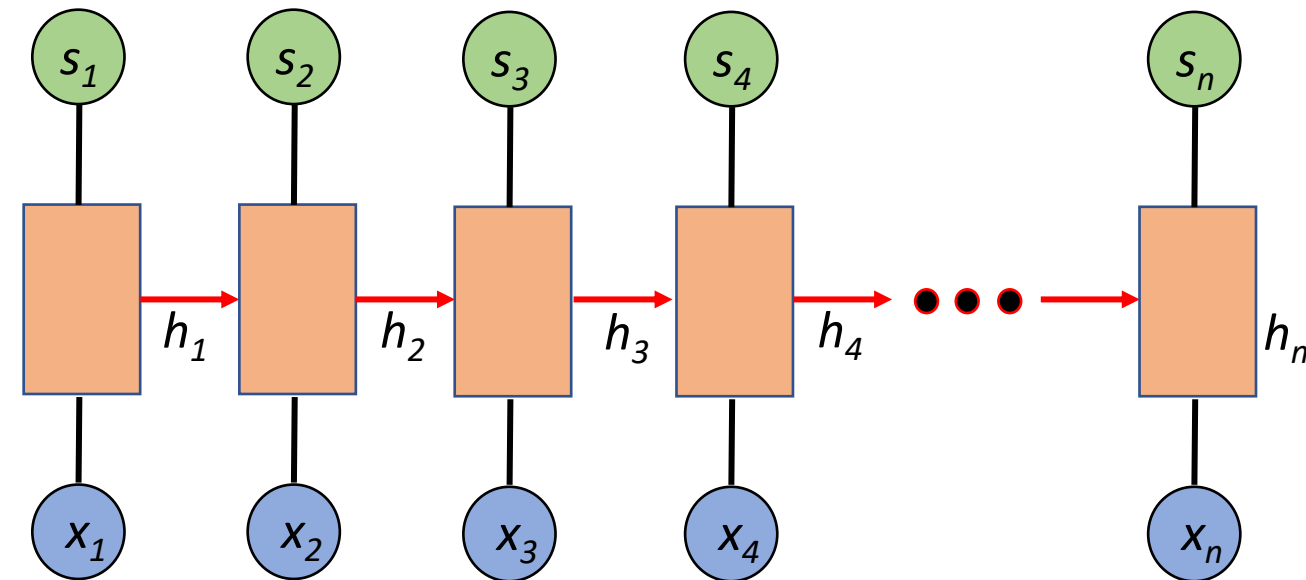
$$s_n = f(x_n, h_{n-1})$$

output

input



Recurrent Neural Network



Solution

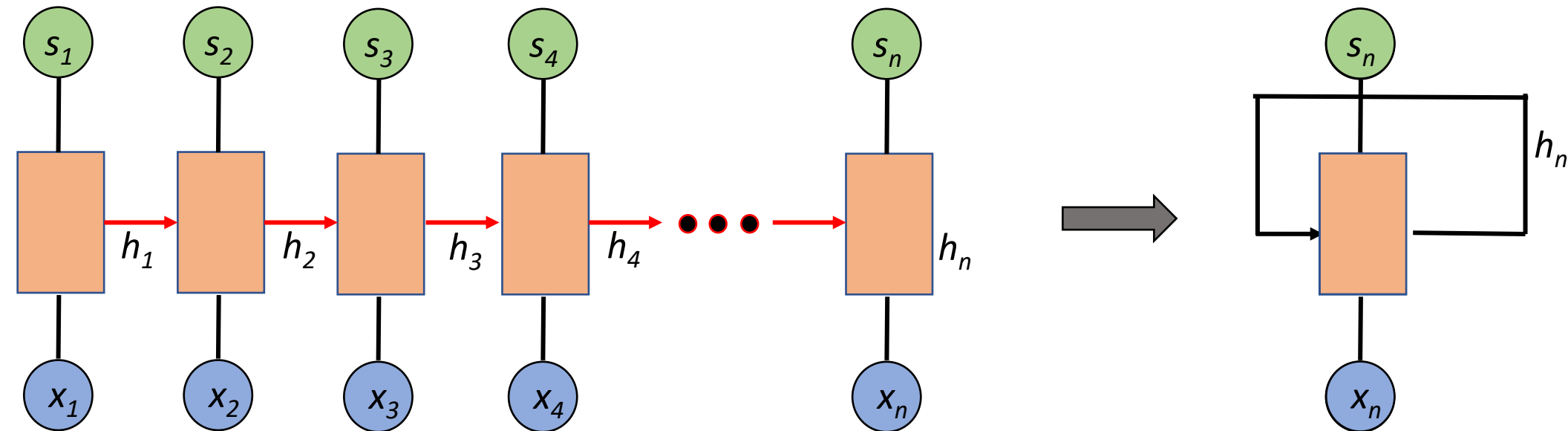
Add recurrent connection

$$s_n = f(x_n, h_{n-1})$$

output input past memory



Recurrent Neural Network

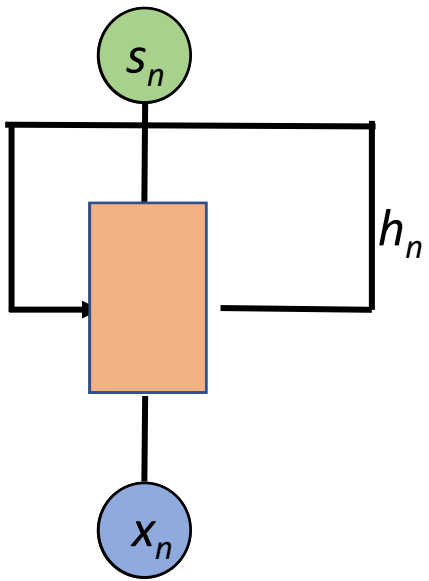


Can be represented
more compactly



Recurrent Neural Network

A sequence of vectors is processed by applying a recurrence formula at each time step.

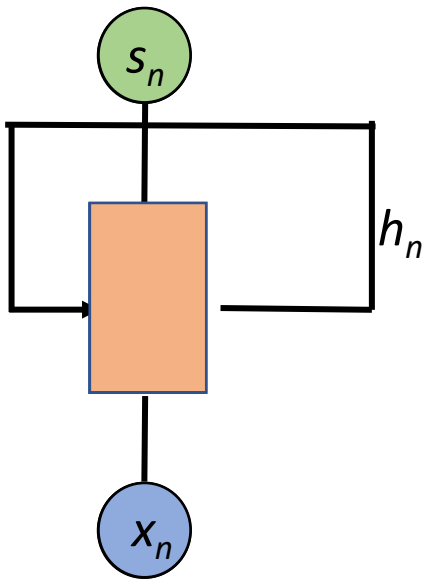




Recurrent Neural Network

A sequence of vectors is processed by applying a recurrence formula at each time step.

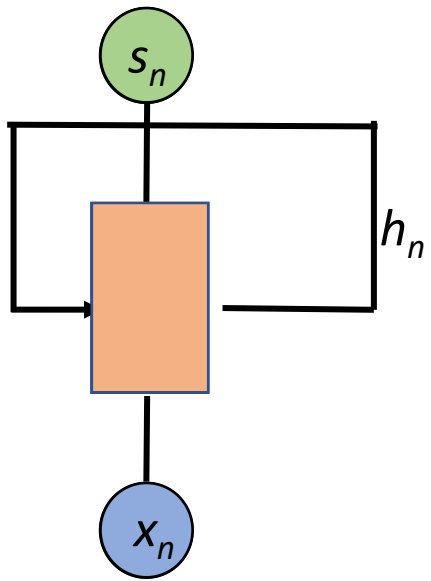
$$h_n = f_W(x_n, h_{n-1})$$



At each time step, h_n is updated as a sequence of input is processed.
The same function and same set of parameters are used in each time step.

Recurrent Neural Network

A sequence of vectors is processed by applying a recurrence formula at each time step.



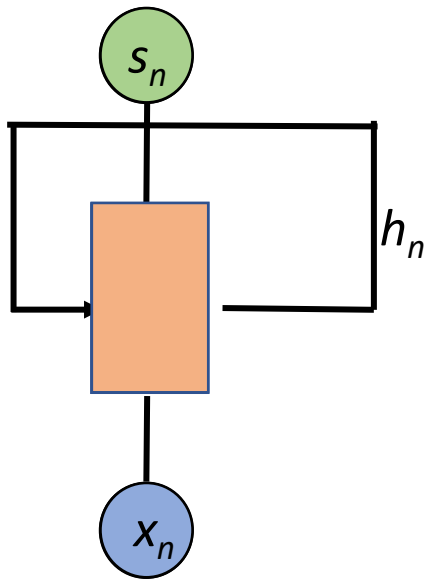
Hidden state

$$h_n = f_W(x_n, h_{n-1})$$



Recurrent Neural Network

A sequence of vectors is processed by applying a recurrence formula at each time step.



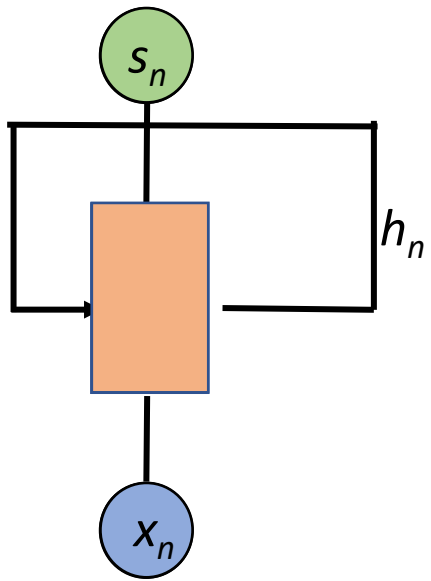
$$h_n = f_W(x_n, h_{n-1})$$

Hidden state

Function with weight W

Recurrent Neural Network

A sequence of vectors is processed by applying a recurrence formula at each time step.



$$h_n = f_W(x_n, h_{n-1})$$

Hidden state

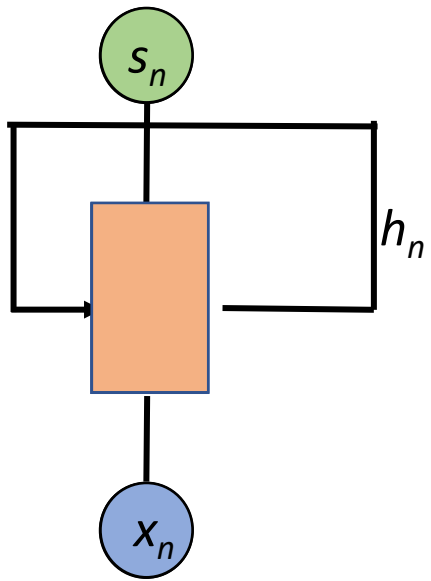
Function with weight W

Input



Recurrent Neural Network

A sequence of vectors is processed by applying a recurrence formula at each time step.

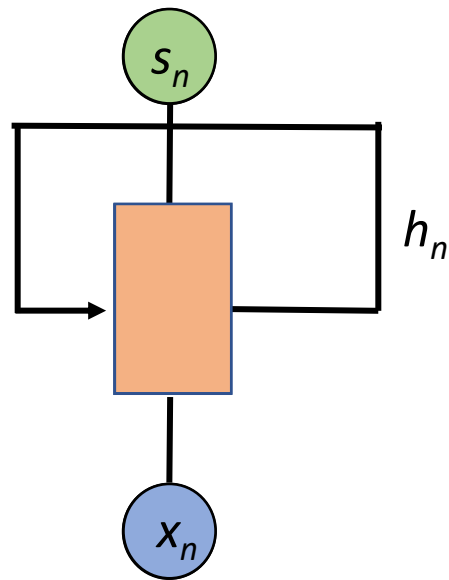


$$h_n = f_W(x_n, h_{n-1})$$

Hidden state → h_n
Function with weight W → f_W
Input → x_n
Old hidden state → h_{n-1}

Recurrent Neural Network

A sequence of vectors is processed by applying a recurrence formula at each time step.



$$h_n = f_W(x_n, h_{n-1})$$

Hidden state

Function with weight W

Input

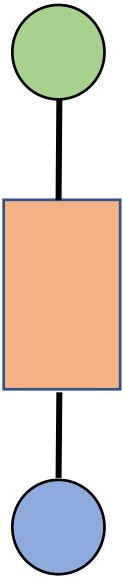
Old hidden state

- Same function is used
- Ensure parameter sharing
- Handles the temporal dependency between sequence



Recurrent Neural Network Architectures

one to one



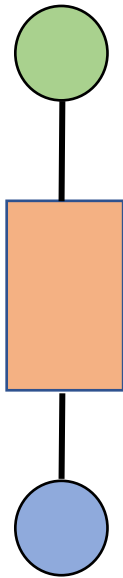
Vanilla NN





Recurrent Neural Network Architectures

one to one



Vanilla NN

one to many

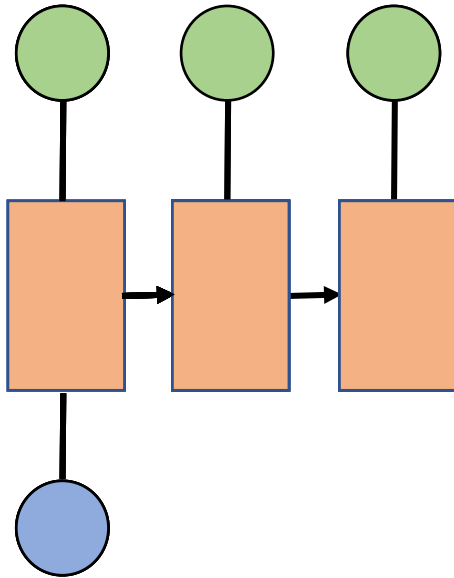
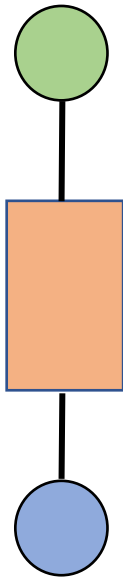


Image
Captioning



Recurrent Neural Network Architectures

one to one



Vanilla NN

one to many

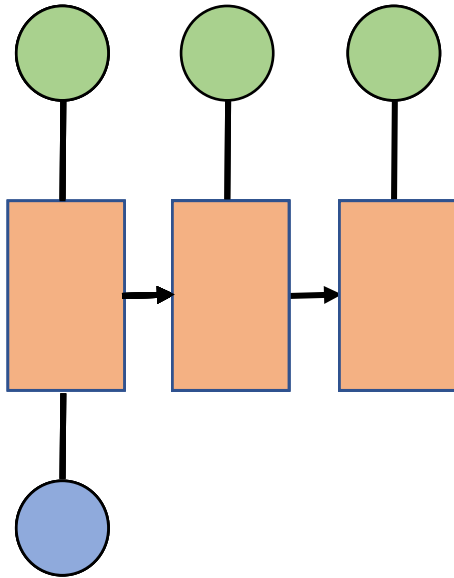
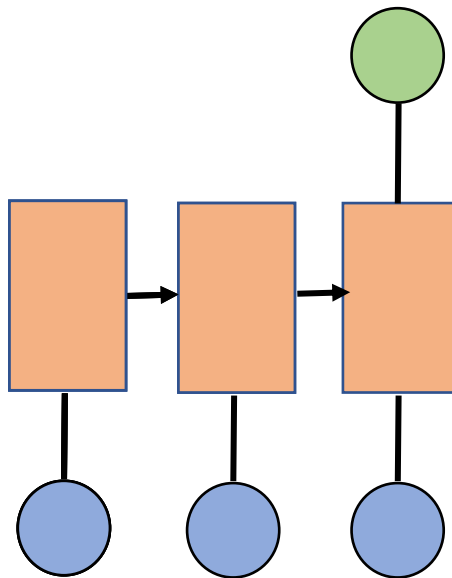


Image
Captioning

many to one

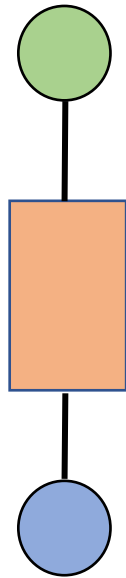


Sentiment
Classification



Recurrent Neural Network Architectures

one to one



Vanilla NN

one to many

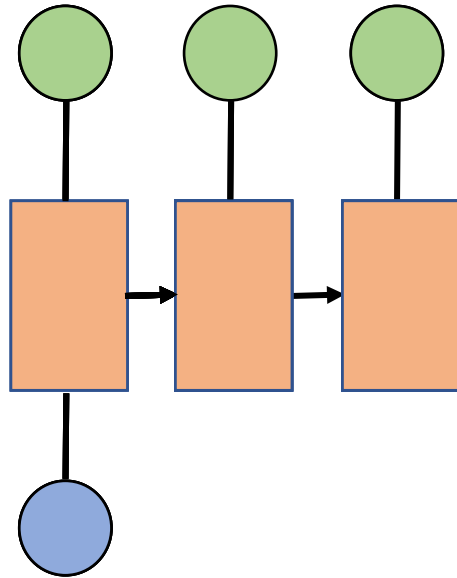
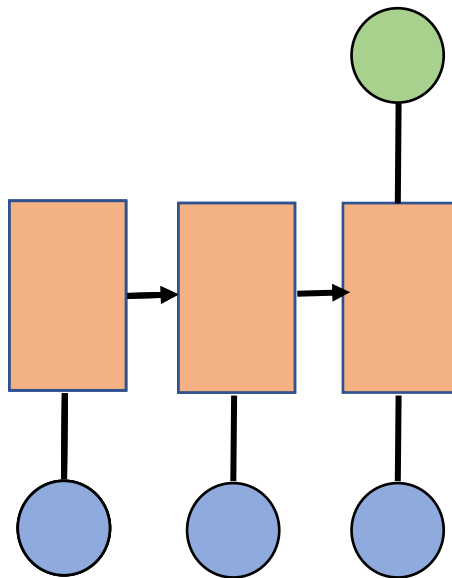


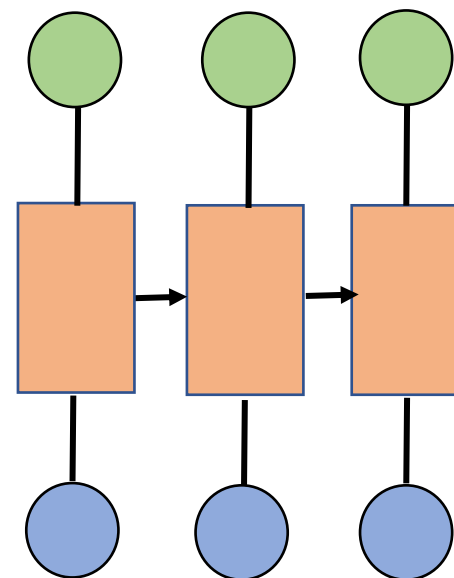
Image
Captioning

many to one



Sentiment
Classification

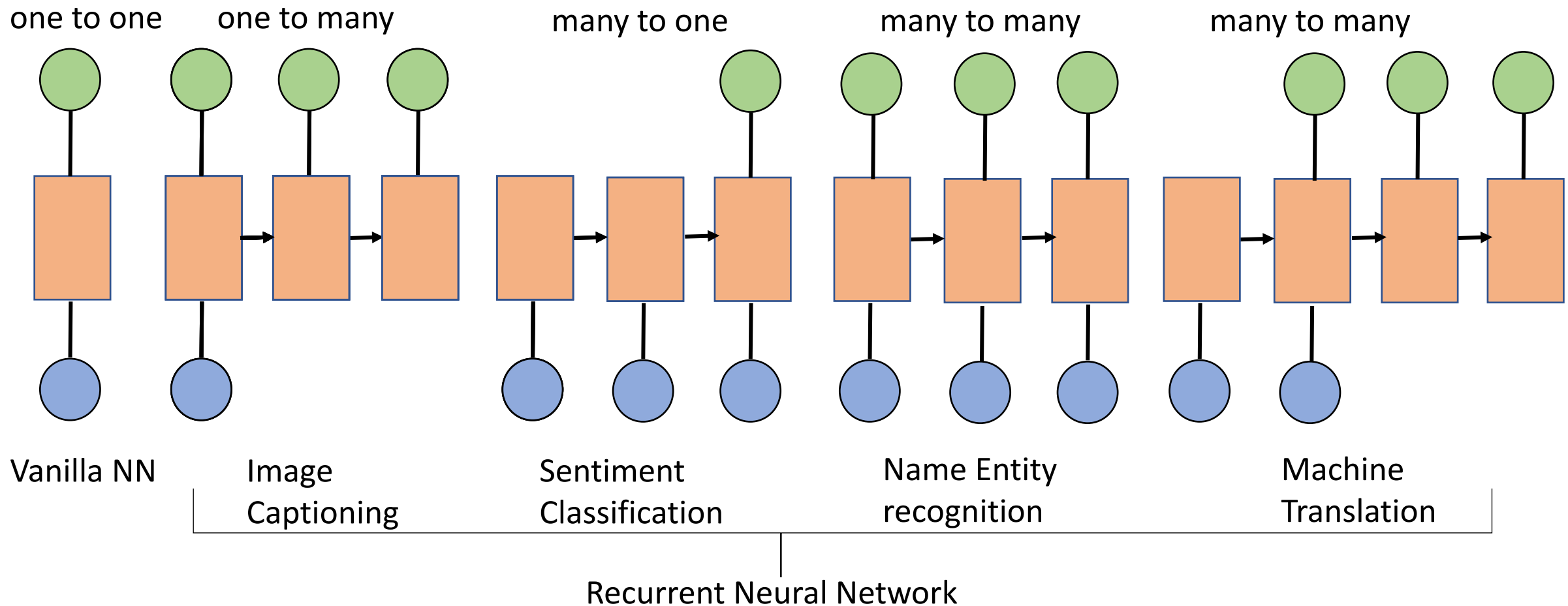
many to many



Name Entity
recognition

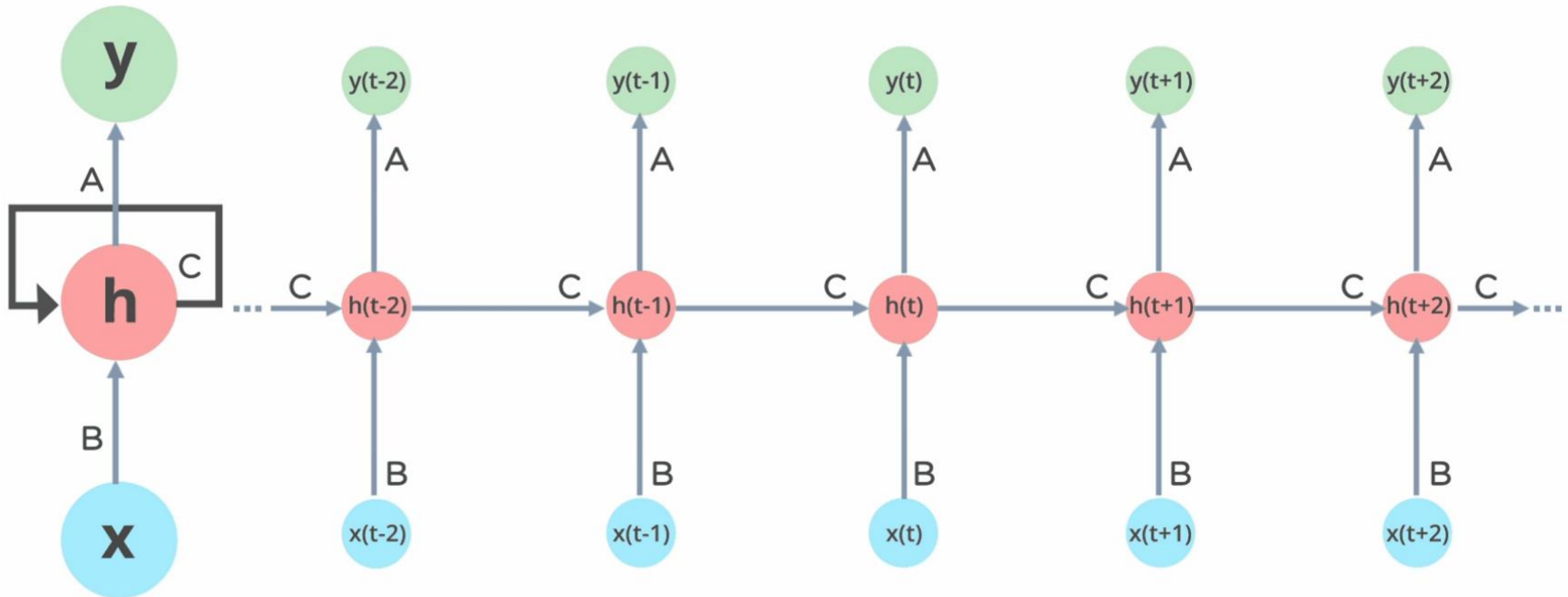


Recurrent Neural Network Architectures





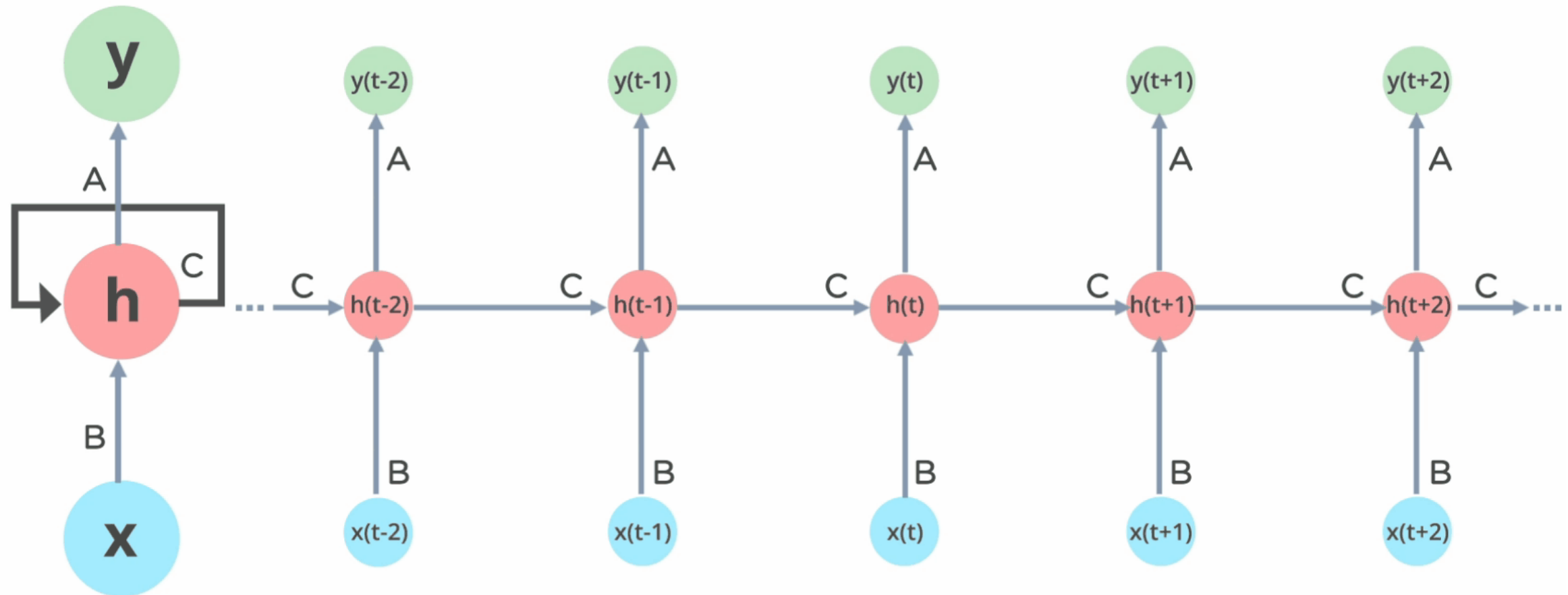
RNN: Forward Propagation



Source: <https://www.simplilearn.com/tutorials/deep-learning-tutorial/rnn>



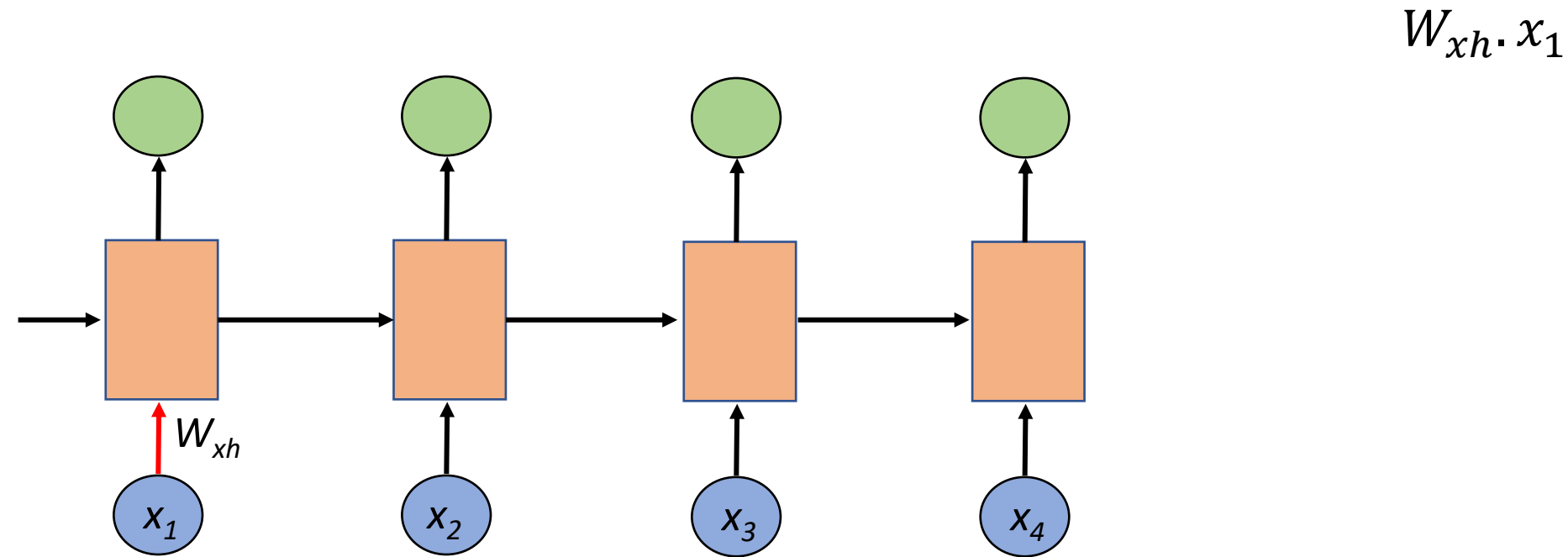
RNN: Forward Propagation



Source: <https://www.simplilearn.com/tutorials/deep-learning-tutorial/rnn>



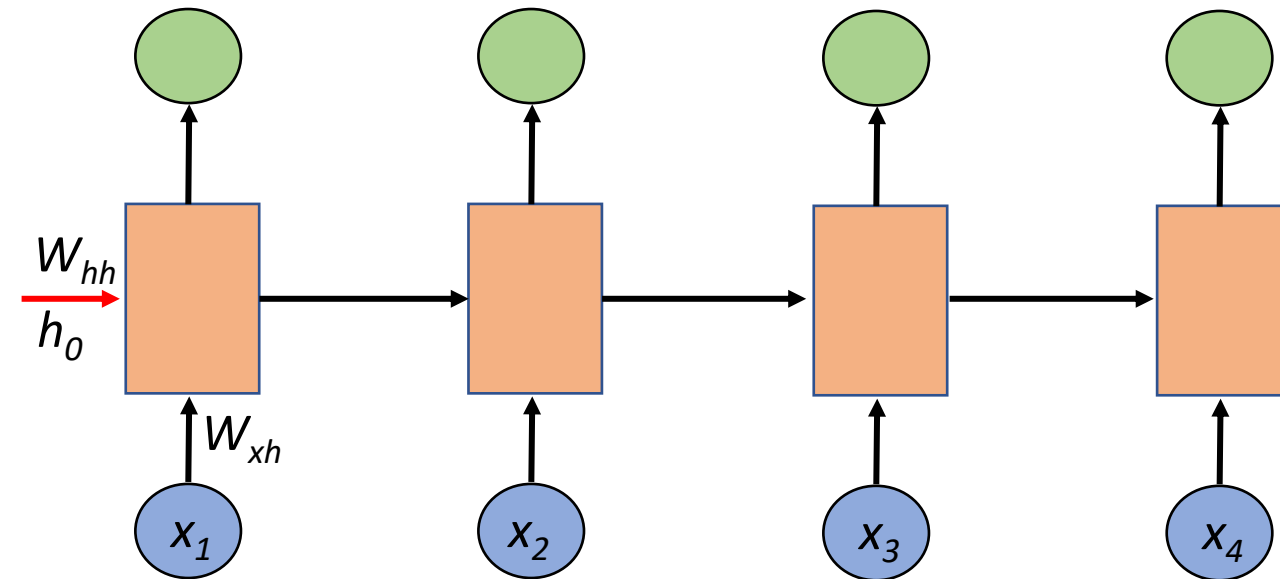
RNN: Forward Propagation





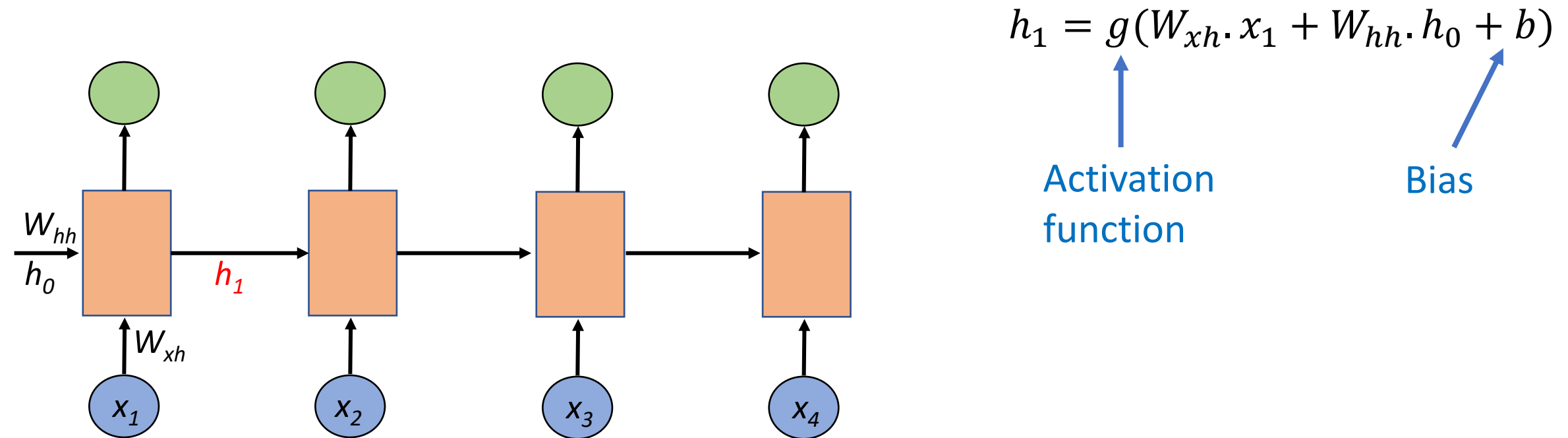
RNN: Forward Propagation

$$W_{xh} \cdot x_1 + W_{hh} \cdot h_0$$



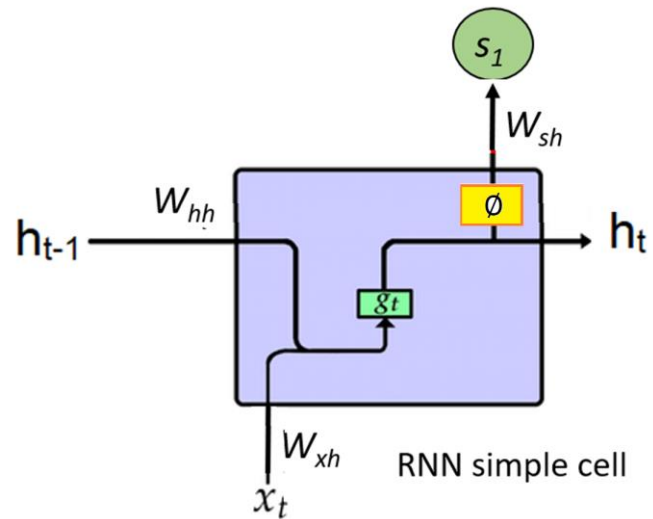


RNN: Forward Propagation



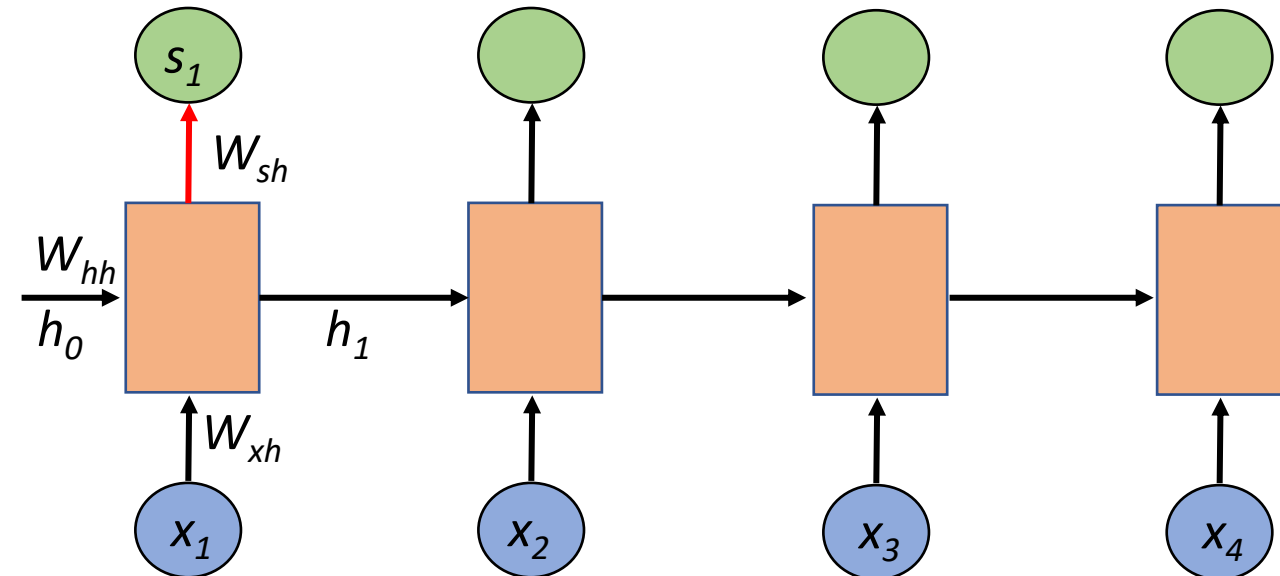


RNN: Forward Propagation



$$h_1 = g(W_{xh} \cdot x_1 + W_{hh} \cdot h_0 + b)$$

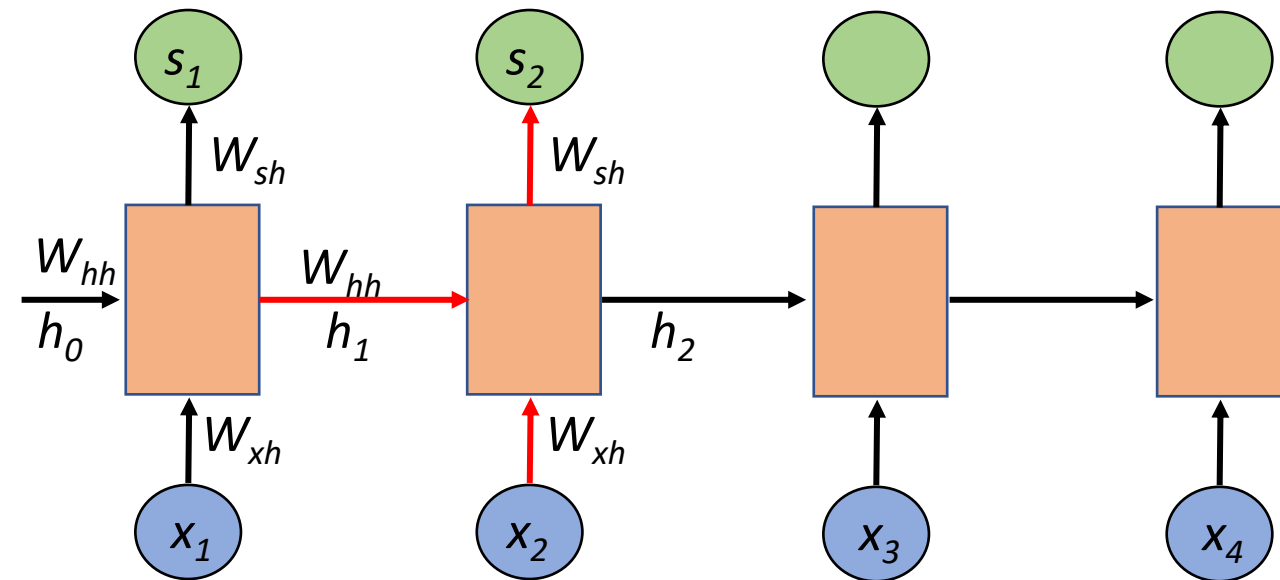
$$s_1 = \phi(W_{sh} \cdot h_1 + c)$$



x_1	Input
h_0	Previous hidden state
h_1	Current hidden state
W_{xh}, W_{hh}, W_{sh}	Shared parameters
g, ϕ	Activation functions
b, c	Biases
s_1	Output state



RNN: Forward Propagation



$$h_1 = g(W_{xh} \cdot x_1 + W_{hh} \cdot h_0 + b)$$

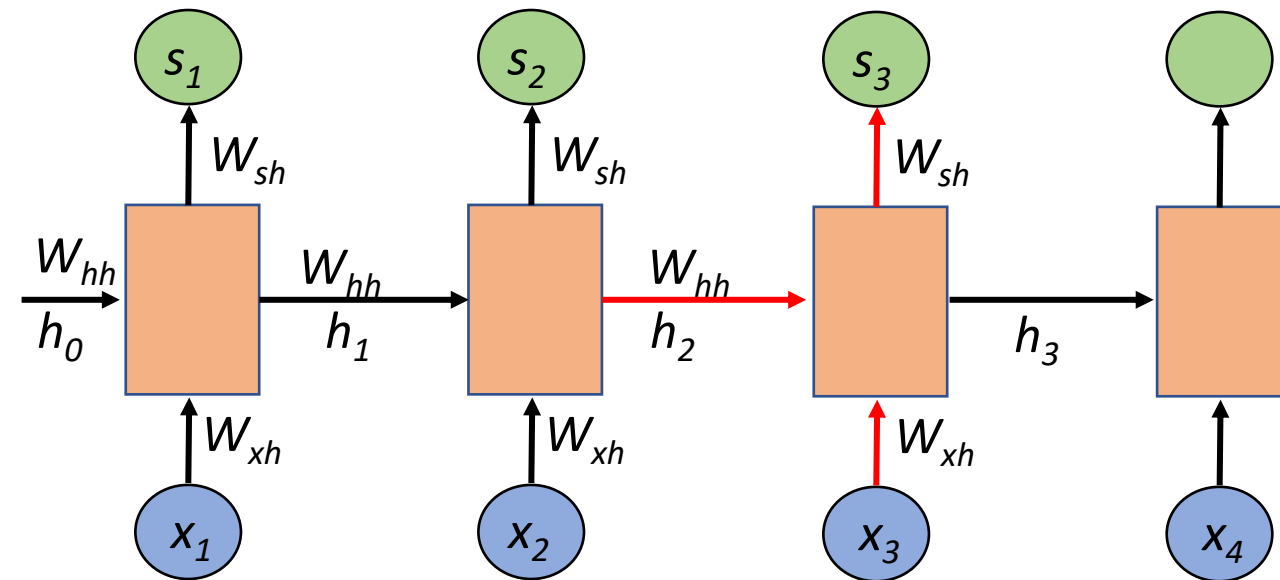
$$s_1 = \phi(W_{sh} \cdot h_1 + c)$$

$$h_2 = g(W_{xh} \cdot x_2 + W_{hh} \cdot h_1 + b)$$

$$s_2 = \phi(W_{sh} \cdot h_2 + c)$$



RNN: Forward Propagation



$$h_1 = g(W_{xh} \cdot x_1 + W_{hh} \cdot h_0 + b)$$

$$s_1 = \phi(W_{sh} \cdot h_1 + c)$$

$$h_2 = g(W_{xh} \cdot x_2 + W_{hh} \cdot h_1 + b)$$

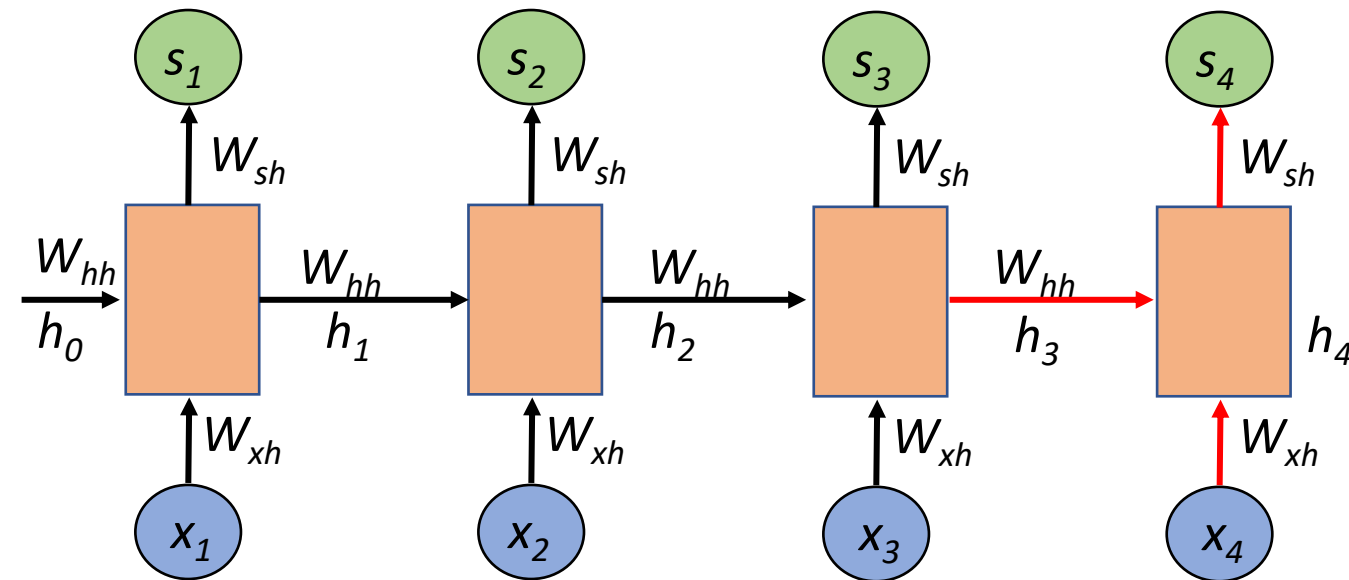
$$s_2 = \phi(W_{sh} \cdot h_2 + c)$$

$$h_3 = g(W_{xh} \cdot x_3 + W_{hh} \cdot h_2 + b)$$

$$s_3 = \phi(W_{sh} \cdot h_3 + c)$$



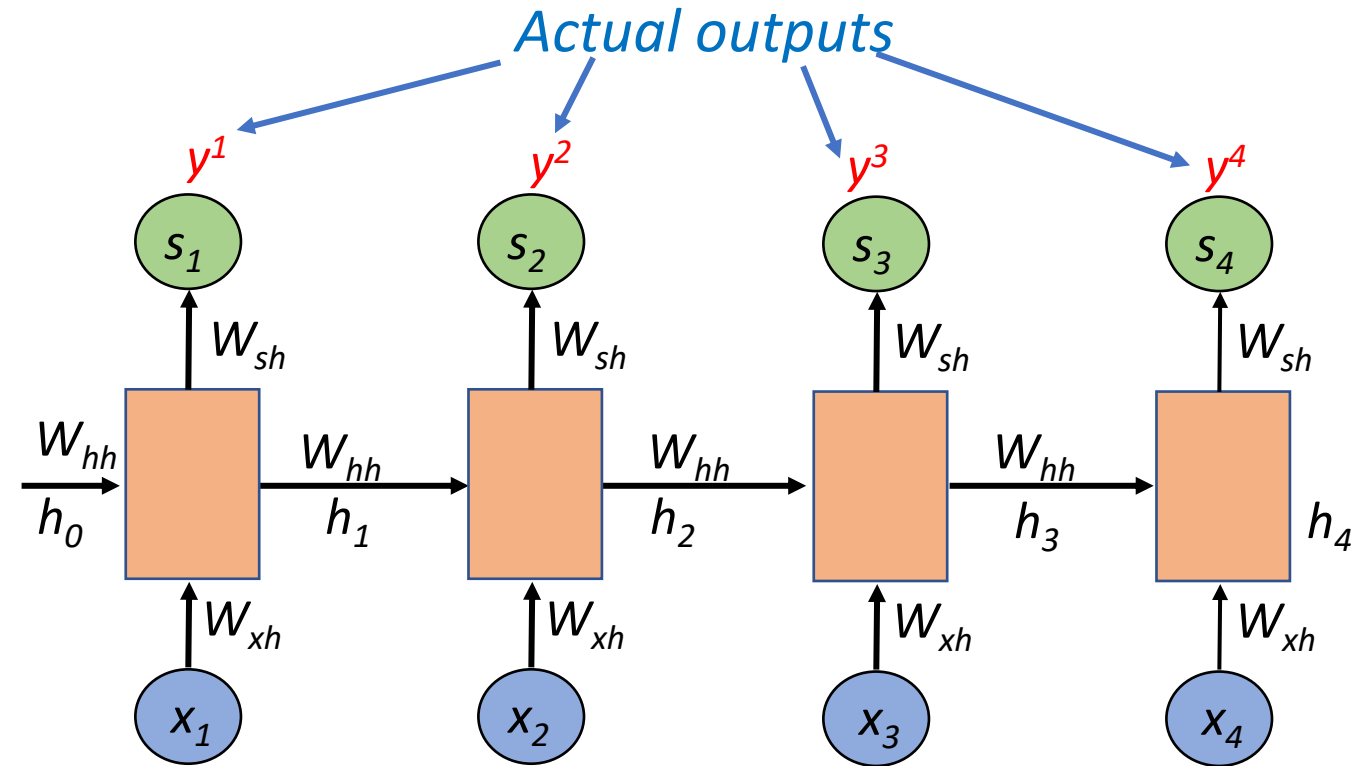
RNN: Forward Propagation



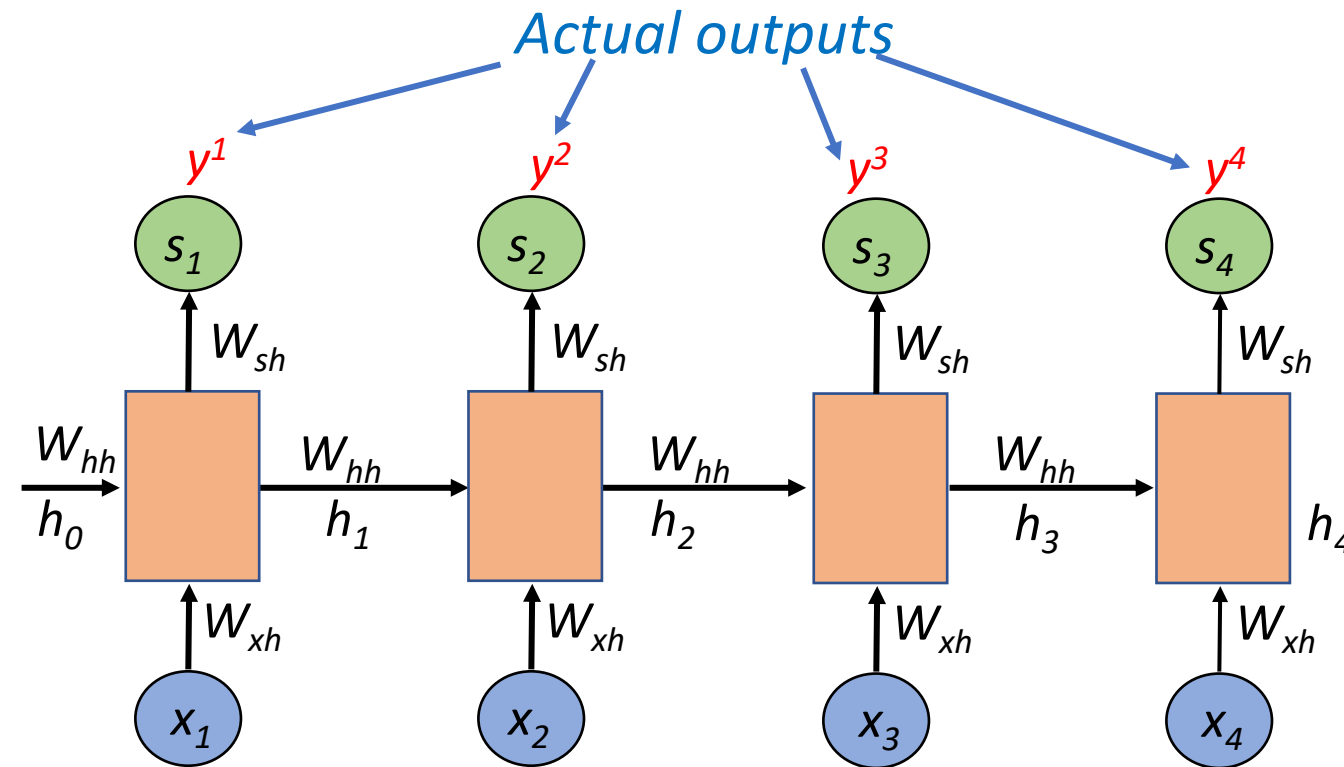
$$\begin{aligned} h_1 &= g(W_{xh} \cdot x_1 + W_{hh} \cdot h_0 + b) \\ s_1 &= \phi(W_{sh} \cdot h_1 + c) \\ h_2 &= g(W_{xh} \cdot x_2 + W_{hh} \cdot h_1 + b) \\ s_2 &= \phi(W_{sh} \cdot h_2 + c) \\ h_3 &= g(W_{xh} \cdot x_3 + W_{hh} \cdot h_2 + b) \\ s_3 &= \phi(W_{sh} \cdot h_3 + c) \\ h_4 &= g(W_{xh} \cdot x_4 + W_{hh} \cdot h_3 + b) \\ s_4 &= \phi(W_{sh} \cdot h_4 + c) \end{aligned}$$

Weight matrix W_{xh} , W_{hh} and W_{sh} remains same throughout the forward propagation, thus ensuring parameter sharing

RNN: Back Propagation Through Time (BPTT)



RNN: Back Propagation Through Time (BPTT)



Loss calculation:

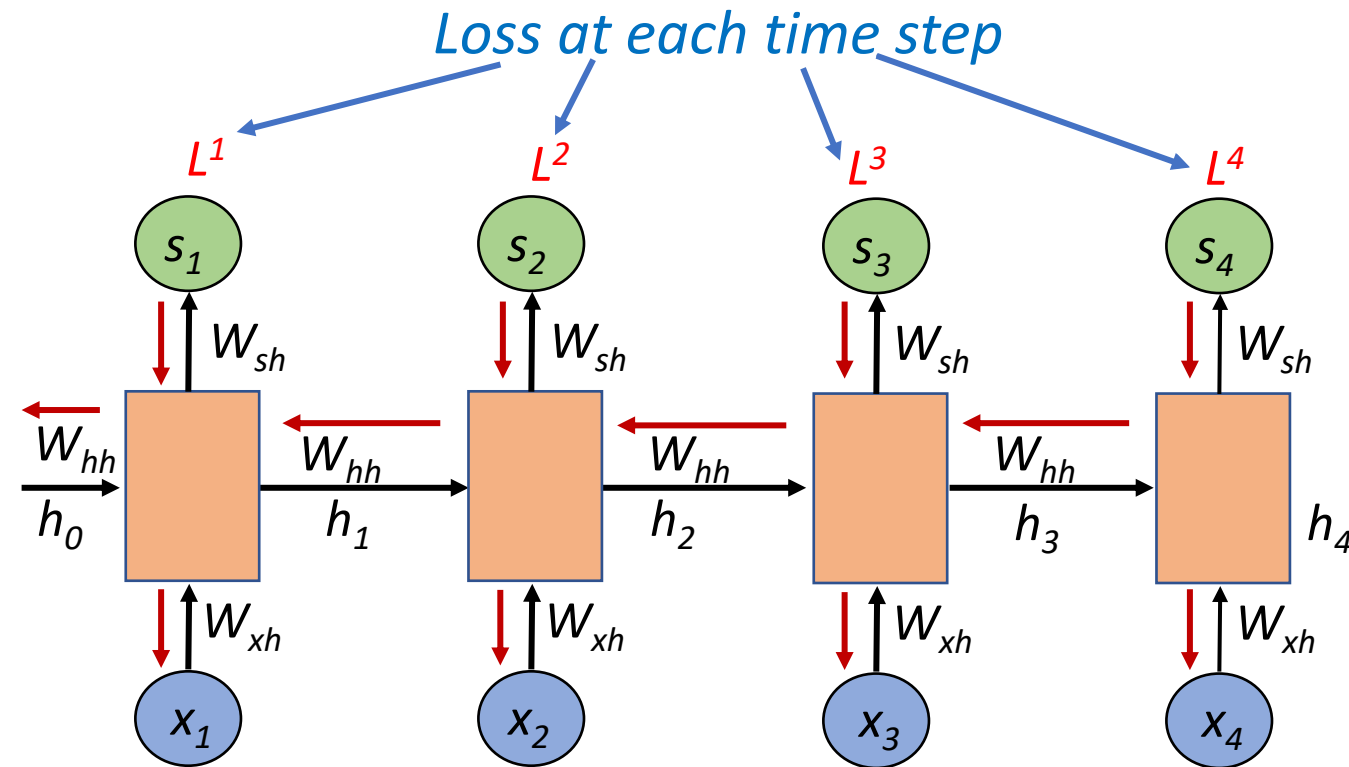
$$\text{Loss } L = \mathcal{L}(y_1, s_1) + \mathcal{L}(y_2, s_2) + \mathcal{L}(y_3, s_3) + \mathcal{L}(y_4, s_4)$$

$\mathcal{L} = \text{Loss function}$

In general:

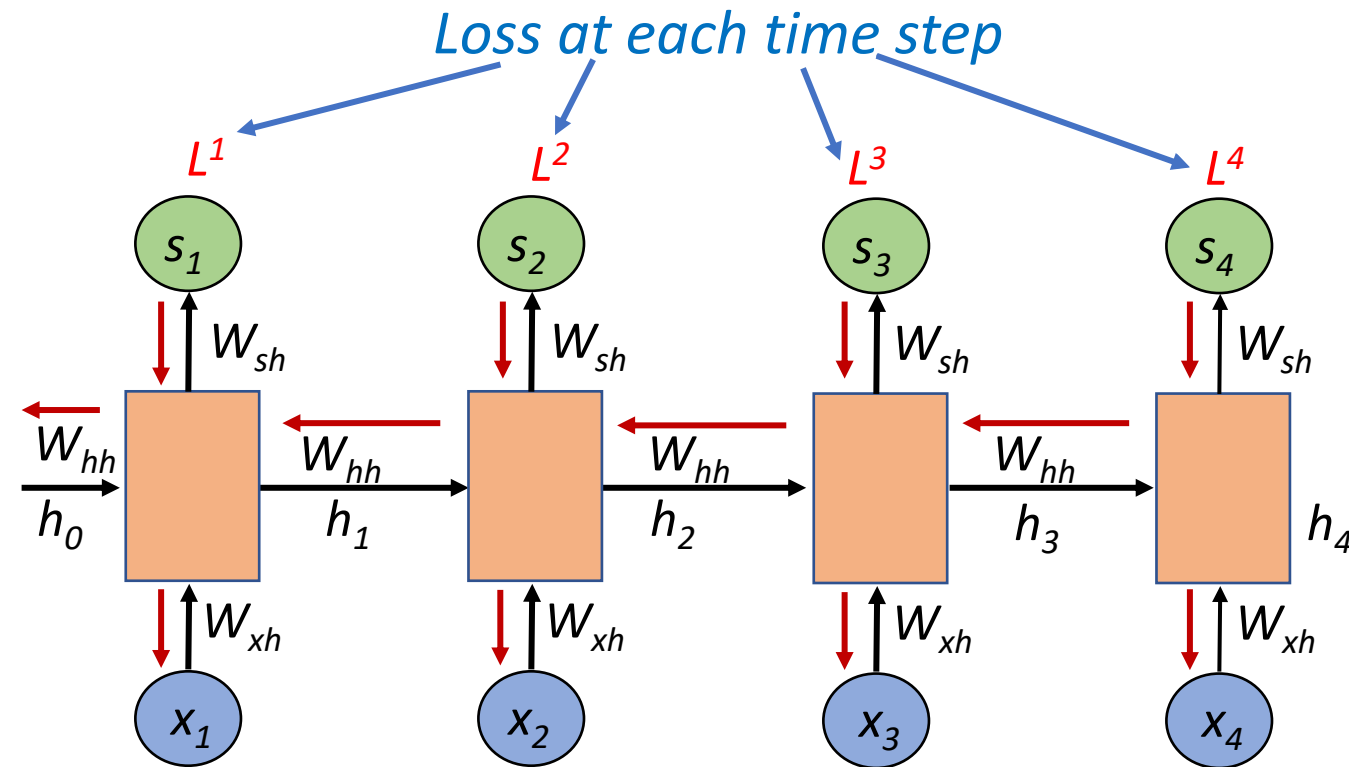
$$L = \sum_{i=1}^n \mathcal{L}(y_i, s_i)$$

RNN: Back Propagation Through Time (BPTT)



Gradient calculation wrt W_{sh} :

RNN: Back Propagation Through Time (BPTT)

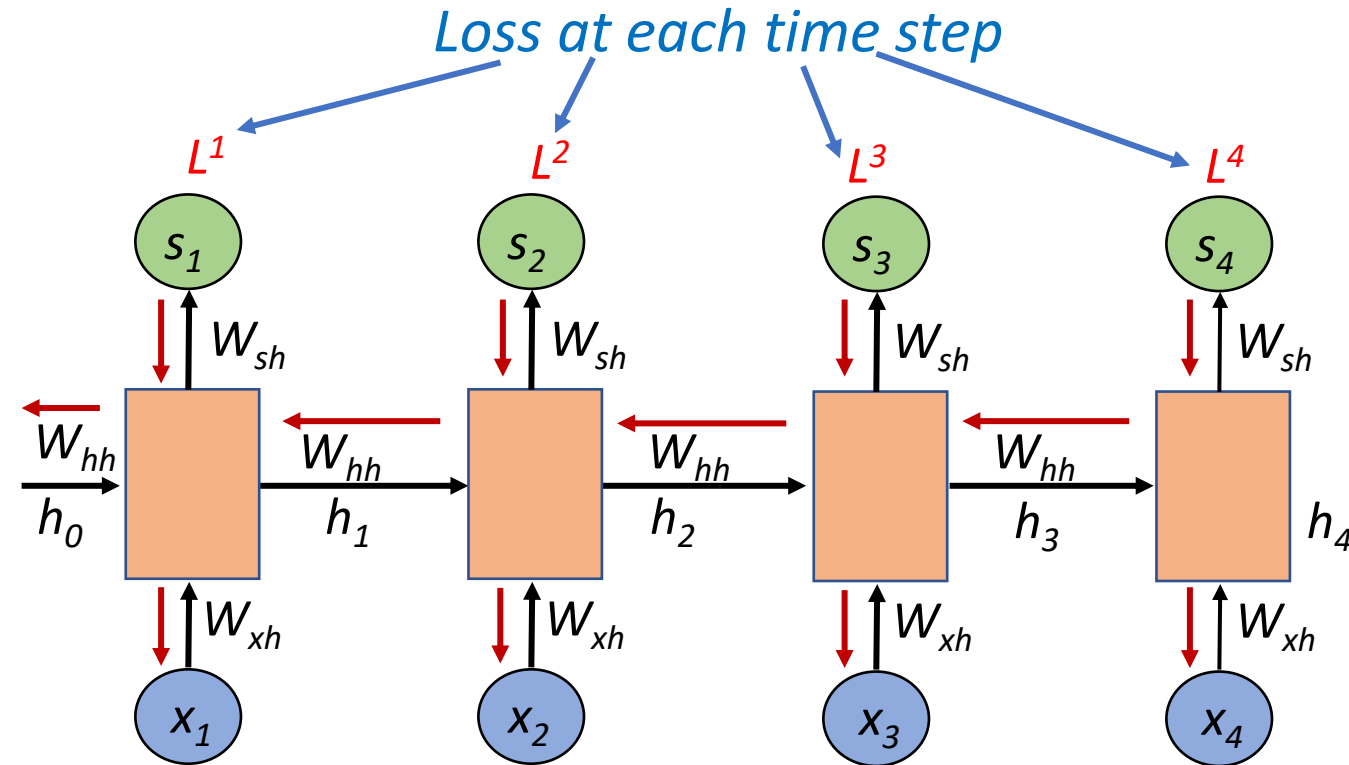


Gradient calculation wrt W_{sh} :

Assumptions:

$$s_4 = W_{sh} \cdot h_4 \text{ and } \mathcal{L} = \text{least square function} = \frac{1}{2} (y_4 - s_4)^2$$

RNN: Back Propagation Through Time (BPTT)



Gradient calculation wrt W_{sh} :

$$\frac{\partial L^4}{\partial W_{sh}} = \frac{\partial L^4}{\partial s_4} \cdot \frac{\partial s_4}{\partial W_{sh}}$$

$$= -(y_4 - s_4) \cdot h_4$$

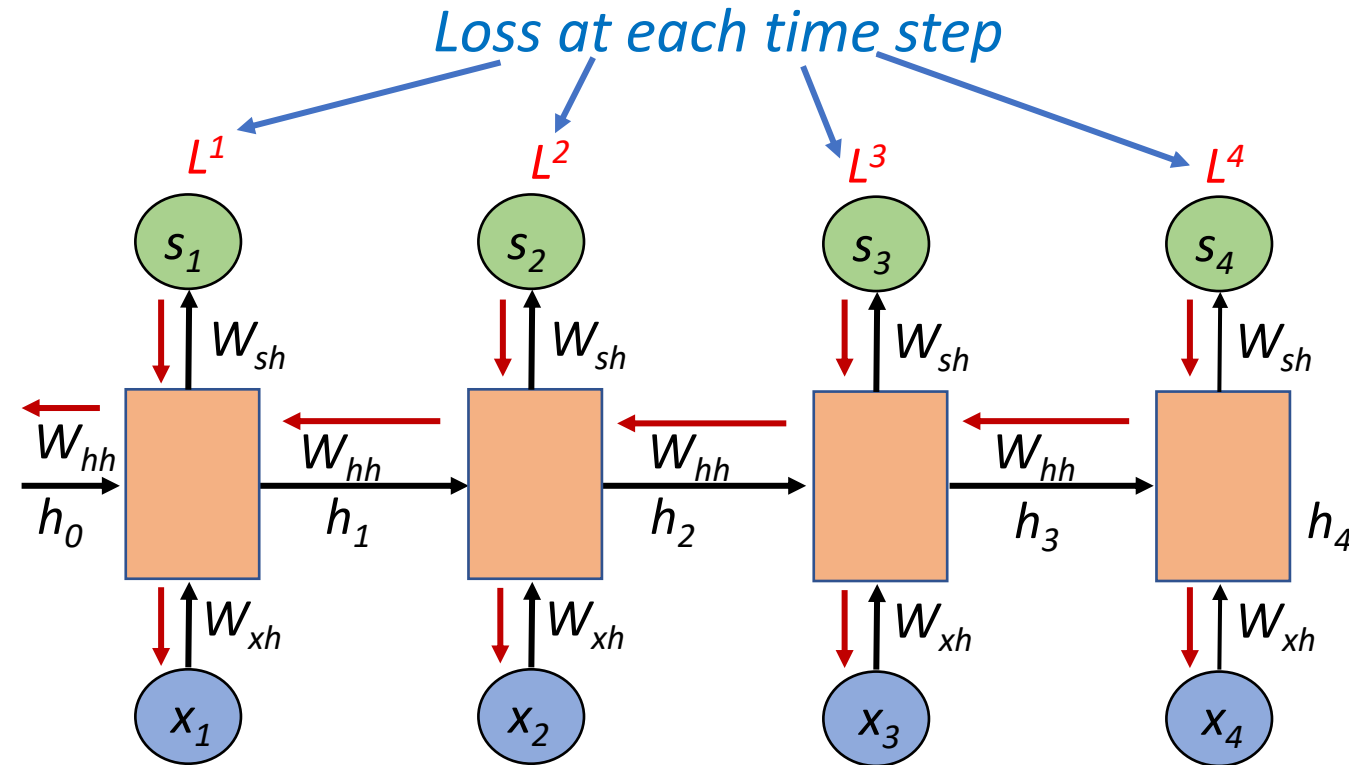
Weight updation wrt W_{sh} :

$$W_{sh} = W_{sh} - \sum_{i=1}^n (y_i - s_i) \cdot h_i$$

Assumptions:

$$s_4 = W_{sh} \cdot h_4 \text{ and } \mathcal{L} = \text{least square function} = \frac{1}{2} (y_4 - s_4)^2$$

RNN: Back Propagation Through Time (BPTT)



Assumptions:

$$s_4 = W_{sh} \cdot h_4 \text{ and } \mathcal{L} = \text{least square function} = \frac{1}{2} (y_4 - s_4)^2$$

Gradient calculation wrt W_{hh} :

$$\frac{\partial L^4}{\partial W_{hh}} = \frac{\partial L^4}{\partial s_4} \cdot \frac{\partial s_4}{\partial h_4} \cdot \frac{\partial h_4}{\partial W_{hh}}$$

$$= -(y_4 - s_4) \cdot W_{sh} \cdot \frac{\partial h_4}{\partial W_{hh}}$$

Now, $h_4 = g(W_{xh} \cdot x_4 + W_{hh} \cdot h_3)$

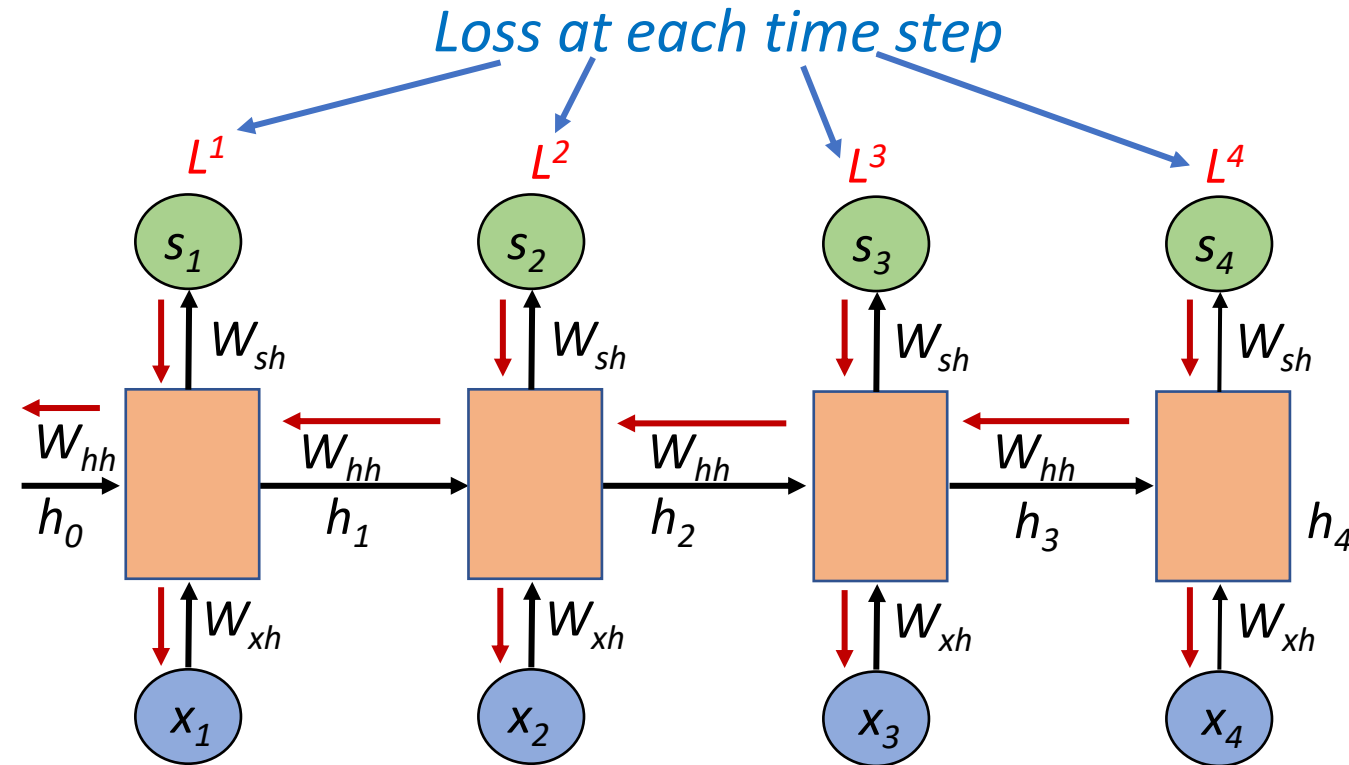
Let, $h_4 = g(z_4)$

$$\text{Then, } \frac{\partial h_4}{\partial W_{hh}} = \frac{\partial h_4}{\partial z_4} \cdot \frac{\partial z_4}{\partial W_{hh}}$$

$$= g'(z_4) \cdot [h_3 + W_{hh} \frac{\partial h_3}{\partial W_{hh}}]$$

$$= g'(z_4) \cdot [h_3 + W_{hh} (h_2 + W_{hh} \frac{\partial h_2}{\partial W_{hh}})]$$

RNN: Back Propagation Through Time (BPTT)



Gradient calculation wrt W_{hh} :

$$\frac{\partial L^4}{\partial W_{hh}} = -(y_4 - s_4) \cdot W_{sh} \cdot g'(z_4) \cdot [h_3 + W_{hh} (h_2 + W_{hh} \frac{\partial h_2}{\partial W_{hh}})]$$

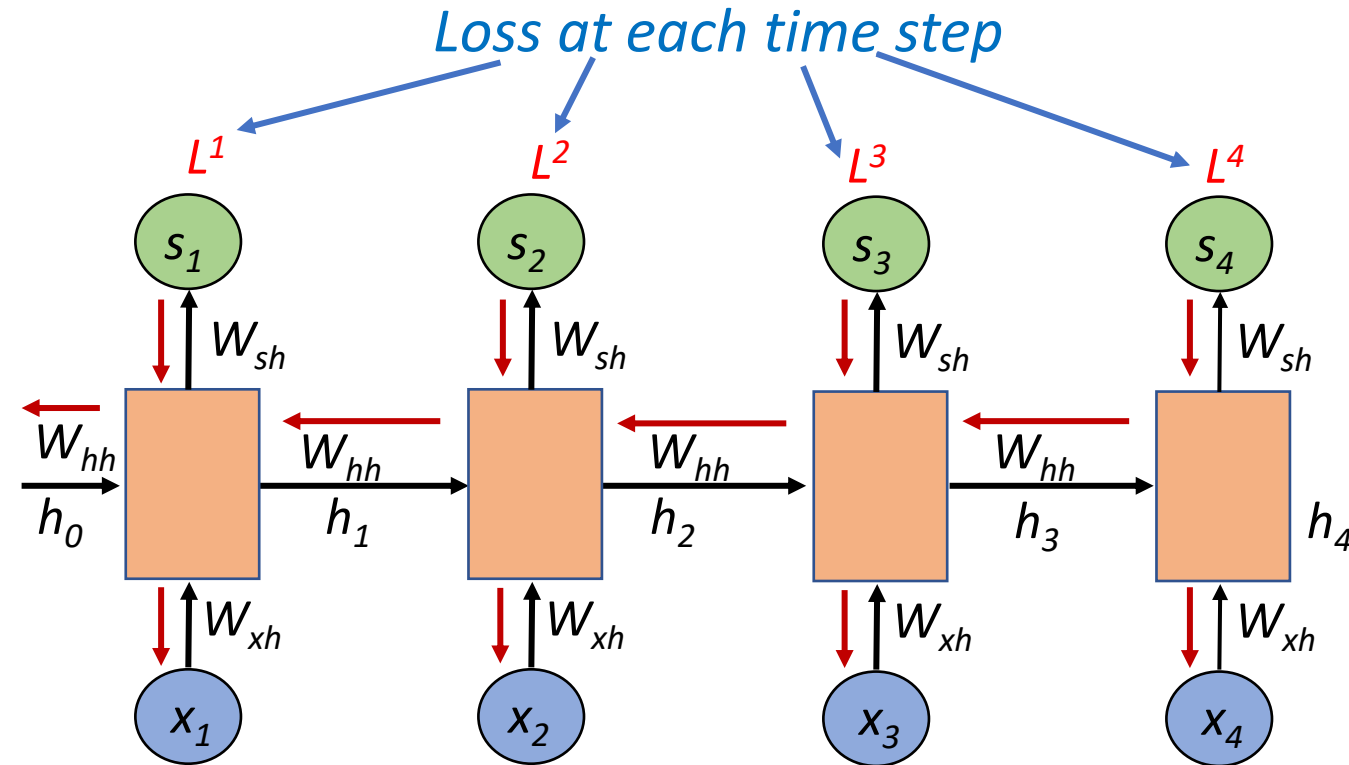
Weight updation wrt W_{hh} :

$$W_{hh} = W_{hh} - \sum_{i=1}^n \frac{\partial L^i}{\partial W_{hh}}$$

Assumptions:

$$s_4 = W_{sh} \cdot h_4 \text{ and } \mathcal{L} = \text{least square function} = \frac{1}{2} (y_4 - s_4)^2$$

RNN: Back Propagation Through Time (BPTT)



Gradient calculation wrt W_{xh} :

$$\frac{\partial L^4}{\partial W_{xh}} = \frac{\partial L^4}{\partial s_4} \cdot \frac{\partial s_4}{\partial h_4} \cdot \frac{\partial h_4}{\partial W_{xh}}$$

$$= -(y_4 - s_4) \cdot W_{sh} \cdot \frac{\partial h_4}{\partial W_{xh}}$$

Now, $h_4 = g(W_{xh} \cdot x_4 + W_{hh} \cdot h_3)$

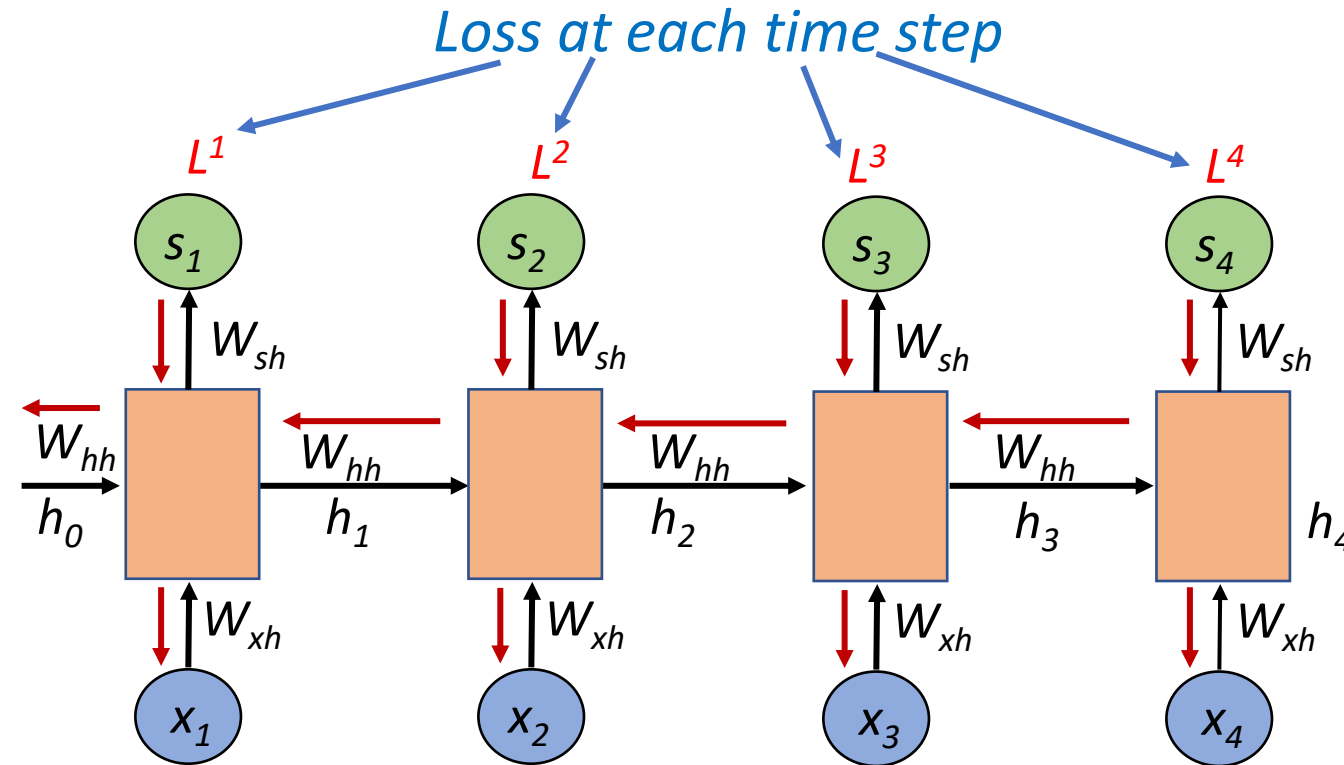
Simply, $h_4 = g(z_4)$

$$\text{Then, } \frac{\partial h_4}{\partial W_{xh}} = \frac{\partial h_4}{\partial z_4} \cdot \frac{\partial z_4}{\partial W_{xh}}$$

$$= g'(z_4) \cdot [x_4 + \frac{\partial W_{hh} \cdot h_3}{\partial W_{xh}}]$$

$$s_4 = W_{sh} \cdot h_4 \text{ and } \mathcal{L} = \text{least square function} = \frac{1}{2} (y_4 - s_4)^2 = g'(z_4) \cdot [x_4 + W_{hh} \cdot g'(z_3) \cdot \frac{\partial z_3}{\partial W_{xh}}]$$

RNN: Back Propagation Through Time (BPTT)



Gradient calculation wrt W_{xh} :

$$\frac{\partial L^4}{\partial W_{xh}} = -(y_4 - s_4)W_{sh} \cdot g' \cdot [x_4 + W_{hh} \cdot g'(z_2) \cdot \frac{\partial z_2}{\partial W_{xh}} \dots]$$

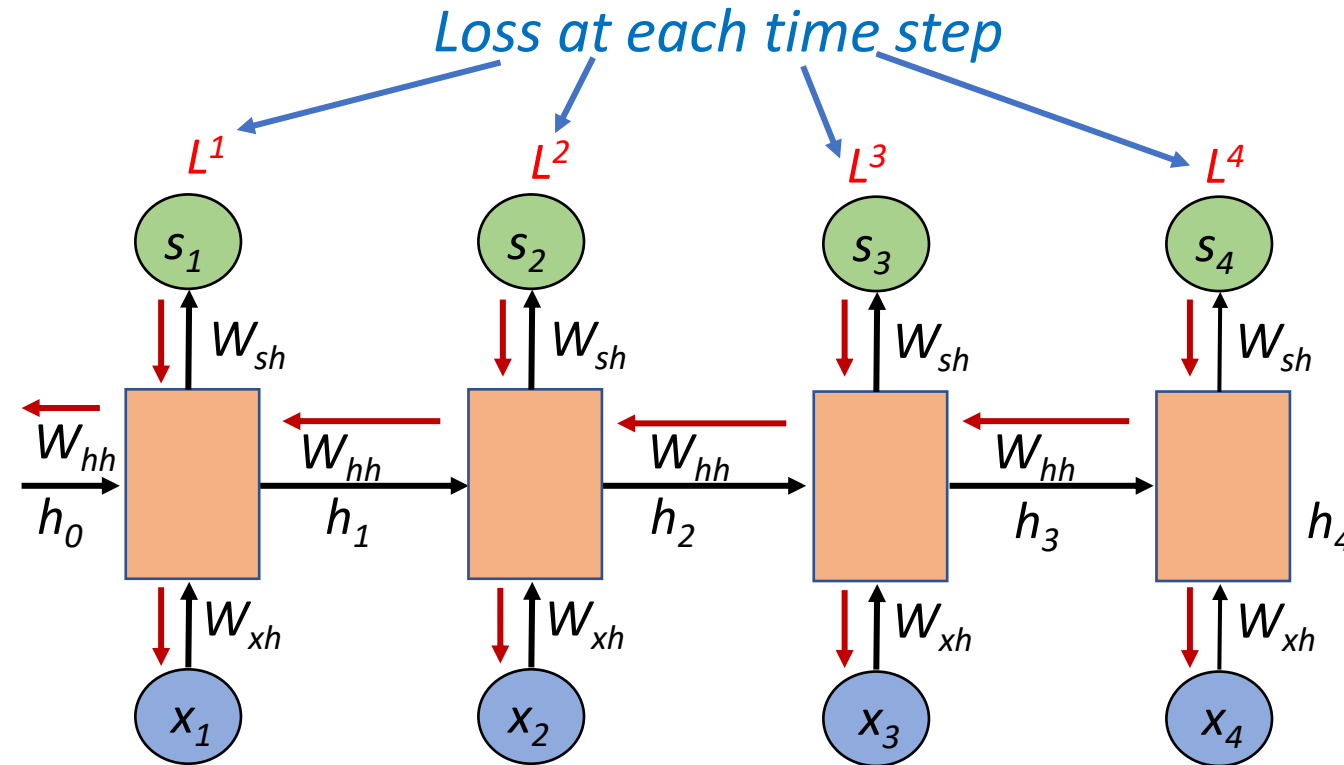
Weight updation wrt W_{xh} :

$$W_{xh} = W_{xh} - \sum_{i=1}^n \frac{\partial L^i}{\partial W_{xh}}$$

Assumptions:

$$s_4 = W_{sh} \cdot h_4 \text{ and } \mathcal{L} = \text{least square function} = \frac{1}{2} (y_4 - s_4)^2$$

RNN: Back Propagation Through Time (BPTT)



Gradient calculation wrt W_{xh} :

$$\frac{\partial L^4}{\partial W_{xh}} = -(y_4 - s_4) W_{sh} \cdot g' \cdot [x_4 + W_{hh} \cdot g'(z_2) \cdot \frac{\partial z_2}{\partial W_{xh}} \dots]$$

Weight updation wrt W_{xh} :

$$W_{xh} = W_{xh} - \sum_{i=1}^n \frac{\partial L^i}{\partial W_{xh}}$$

`tf.keras.layers.SimpleRNN(rnn_units)`



Assumptions:

$$s_4 = W_{sh} \cdot h_4 \text{ and } \mathcal{L} = \text{least square function} = \frac{1}{2} (y_4 - s_4)^2$$



Limitations of RNN

- Gradient calculation involves many factors of weights and contribution of activation function.





Limitations of RNN

- Gradient calculation involves many factors of weights and contribution of activation function.
- This may lead to:

Exploding Gradient



Limitations of RNN

- Gradient calculation involves many factors of weights and contribution of activation function.
- This may lead to:

Exploding Gradient

Vanishing Gradient

Limitations of RNN

- Gradient calculation involves many factors of weights and contribution of activation function.
- This may lead to:

Exploding Gradient

Vanishing Gradient

$$\frac{\partial L}{\partial W} \gg 1$$

→ *resulting in NAN values*



Limitations of RNN

- Gradient calculation involves many factors of weights and contribution of activation function.
- This may lead to:

Exploding Gradient

$$\frac{\partial L}{\partial W} \gg 1$$

→ resulting in NAN values

Vanishing Gradient

$$\frac{\partial L}{\partial W} \ll 1$$

→ resulting in 0 values



Limitations of RNN

- Gradient calculation involves many factors of weights and contribution of activation function.
- This may lead to:

Exploding Gradient

Vanishing Gradient

- make learning unstable



Limitations of RNN

- Gradient calculation involves many factors of weights and contribution of activation function.
- This may lead to:

Exploding Gradient

- make learning unstable

Vanishing Gradient

- Short term dependencies

“the stars shine in the ?” → sky (*RNN works good here*)

- Long term dependencies

“I grew up in Spain..... I speak fluent Spanish”. (*Difficult for RNN to remember as gap increases*)



Possible Solutions

Exploding Gradient

Vanishing Gradient

❑ Gradient clipping

```
# inside the optimizer we are doing clipping  
optimizer=tf.keras.optimizers.SGD(clipvalue=0.5)
```



Possible Solutions

Exploding Gradient

- ☐ Gradient clipping

```
# inside the optimizer we are doing clipping  
optimizer=tf.keras.optimizers.SGD(clipvalue=0.5)
```

Vanishing Gradient

- ☐ Activation function (Relu)
- ☐ Weight initialization (identity matrix)
- ☐ Gated cells (LSTM,GRU,etc)